PLOSHA-RMFR: Predictive Load-Sharing Hierarchical Aggregation with Risk-Aware Multi-Layer Fault Recovery for Secure IIoT Edge–Fog Computing

Somchart Fugkeaw, *Member, IEEE*

***Abstract*—Fog-assisted Industrial Internet of Things (IIoT) environments continuously generate large volumes of latency-sensitive sensor data that require efficient and reliable aggregation. However, existing aggregation frameworks commonly rely on static aggregation windows, flat aggregation structures, and reactive fault recovery, resulting in high aggregation-loss exposure, inefficient recovery, and limited adaptability under dynamic workloads and failures. To address these challenges, this paper proposes PLOSHA-RMFR, a secure and self-adaptive fog aggregation framework that integrates **Predictive Load-Sharing Hierarchical Aggregation (PLOSHA) with Risk-Aware Multi-Layer Fault Recovery (RMFR). PLOSHA employs lightweight predictive estimation to evaluate future processing capacity, failure exposure, operational risk, and node reliability, enabling adaptive micro-slot partitioning and hierarchical encrypted aggregation using TEE-assisted Paillier homomorphic encryption. By localizing aggregation loss to affected micro-slots, the proposed hierarchy significantly reduces recovery overhead compared with conventional flat aggregation approaches. RMFR further introduces a progressive recovery mechanism that combines secure predictive load sharing, selective micro-slot recovery, and reliability-aware failover to maintain aggregation continuity under adverse conditions. In addition, an Adaptive Feedback Learning and Threshold Optimization (AFLTO) mechanism continuously adjusts aggregation and recovery parameters according to observed operational performance, enabling closed-loop self-optimization. Experimental results demonstrate that PLOSHA-RMFR effectively reduces aggregation-loss exposure and recovery latency while improving aggregation completeness and system reliability under heterogeneous workload and fault scenarios. These results demonstrate the framework's suitability as a secure, scalable, and fault-resilient aggregation solution for next-generation IIoT edge–fog environments.**

***Index Terms*—Industrial Internet of Things, Edge–fog computing, Fault tolerance, Load balancing, Trusted execution environment, Paillier encryption, Key management**

I. Introduction

The Industrial Internet of Things (IIoT) has become a key enabler of Industry 4.0, supporting predictive maintenance, autonomous control, intelligent monitoring, and real-time analytics across modern industrial environments [1]. Large-scale IIoT deployments continuously generate massive volumes of latency-sensitive sensor data from heterogeneous devices such as vibration, temperature, and pressure sensors. Many industrial applications, including smart manufacturing, chemical plants, and power grids, require timely detection of abnormal conditions to avoid operational disruption and safety hazards. Consequently, IIoT infrastructures demand low-latency processing, high availability, efficient resource utilization, and strong confidentiality guarantees.

Edge–fog computing has emerged as an effective solution by moving computation closer to data sources, thereby reducing communication delay and cloud bandwidth consumption [3], [31]. In such architectures, fog nodes perform local preprocessing, aggregation, and analytics before forwarding summarized information to cloud services. Despite these advantages, existing fog-based IIoT systems still face significant challenges in achieving scalability, resilience, and privacy-preserving processing simultaneously.

First, heterogeneous fog environments often suffer from resource imbalance because fog nodes possess different processing capacities, memory resources, and network bandwidths. Static sensor-to-fog assignments may overload constrained nodes while leaving capable nodes underutilized [4], [8]. Although optimization-based, learning-based, and decentralized scheduling methods have been proposed [12], [15], [22], most rely on centralized coordination, offline training, or reactive workload redistribution, limiting adaptability under dynamic workloads.

Second, fault tolerance remains largely reactive. Existing solutions predominantly employ replication, checkpointing, or task resubmission mechanisms [5], [18]–[20]. While effective for service continuity, these approaches incur substantial storage, communication, and recovery overhead. Moreover, load balancing and fault recovery are typically treated independently, despite overload, communication disruption, and node failures frequently occurring together in practical IIoT deployments.

Third, privacy-preserving aggregation remains challenging in zero-trust fog environments. Many existing systems require access to plaintext sensor readings during aggregation, exposing sensitive operational information [33]. Homomorphic aggregation techniques such as Paillier encryption enable computation over encrypted data and preserve confidentiality throughout the aggregation process [7], [24], [34]. However, most existing schemes employ flat aggregation structures and fixed aggregation windows, causing communication and storage costs to grow linearly with the number of collected readings.

More critically, current encrypted aggregation frameworks rarely integrate aggregation efficiency with fault resilience.

S. Fugkeaw, Author2, Author3 are with the School of Information, Computer and Communication Technology, Sirindhorn International Institute of Technology (SIIT), Thammasat University, Pathum Thani 12121, Thailand.

Corresponding author: Somchart Fugkeaw (e-mail: somchart@siit.tu.ac.th).

Fixed aggregation windows and static aggregation granularity often result in excessive aggregation-loss exposure when failures occur near the end of an aggregation epoch. Similarly, workload migration is usually triggered only after congestion or failure has occurred, increasing recovery latency and service disruption. As a result, existing solutions provide limited support for simultaneously minimizing aggregation loss, recovery overhead, and processing delay.

Although prior studies have investigated resource management, fault tolerance, secure aggregation, and privacy preservation separately, several important limitations remain. Existing aggregation frameworks lack adaptive aggregation granularity, resulting in significant aggregation-loss exposure under failures. Workload adaptation and fault recovery are generally performed independently and reactively, leading to inefficient resource utilization and delayed recovery. Most fault-tolerant mechanisms rely on costly replication or full-window recovery strategies rather than localized recovery. Furthermore, fixed recovery thresholds cannot adapt to changing workload dynamics and reliability conditions. Finally, secure workload delegation remains difficult because migrated processing often requires access to sensitive aggregation states or cryptographic material.

To address these limitations, this paper proposes PLOSHA-RMFR, a secure and resilient aggregation framework for IIoT edge–fog environments. The framework integrates predictive load sharing, adaptive hierarchical encrypted aggregation, risk-aware fault recovery, and adaptive feedback optimization within a unified closed-loop architecture.

The proposed Predictive Load-Sharing Hierarchical Aggregation (PLOSHA) mechanism dynamically adjusts aggregation granularity according to predicted processing capacity, failure exposure, and node reliability. By partitioning aggregation epochs into adaptive micro-slots and constructing hierarchical encrypted aggregates, PLOSHA localizes aggregation loss and improves aggregation efficiency. To maintain aggregation continuity, the Risk-Aware Multi-Layer Fault Recovery (RMFR) framework employs progressive recovery through predictive delegation, selective micro-slot recovery, and reliability-aware failover. In addition, the Adaptive Feedback Learning and Threshold Optimization (AFLTO) mechanism continuously refines aggregation and recovery thresholds according to observed operational conditions. To preserve confidentiality during aggregation and workload migration, PLOSHA-RMFR incorporates TEE-protected ciphertext transformation and secure fog-scoped delegation without exposing plaintext sensor data or cryptographic material.

The main contributions of this paper are summarized as follows:

- We propose Predictive Load-Sharing Hierarchical Aggregation (PLOSHA), which dynamically adapts aggregation granularity according to predicted capacity, failure exposure, and reliability conditions. By combining adaptive micro-slot partitioning with hierarchical encrypted aggregation, PLOSHA reduces aggregation-loss exposure while preserving privacy-preserving processing efficiency.
- We design a Risk-Aware Multi-Layer Fault Recovery (RMFR) model that integrates predictive delegation, localized micro-slot recovery, and reliability-aware failover within a unified recovery-escalation model, enabling efficient fault-tolerant aggregation with reduced recovery overhead.
- We develop a TEE-protected secure delegation architecture that supports confidential workload migration through enclave-confined state transfer and temporary credential provisioning, preserving aggregation continuity without exposing sensitive data or cryptographic material.
- We introduce an Adaptive Feedback Learning and Threshold Optimization (AFLTO) mechanism that continuously optimizes aggregation and recovery thresholds according to aggregation quality, recovery urgency, and reliability conditions, enabling closed-loop self-adaptation.

II. Related Work

Research related to resilient Industrial Internet of Things (IIoT) edge–fog systems can be broadly classified into three areas: adaptive resource management, fault-tolerant recovery mechanisms, and privacy-preserving aggregation with secure delegation. Although significant progress has been achieved in each area individually, existing solutions rarely integrate these capabilities within a unified framework capable of simultaneously optimizing efficiency, resilience, and confidentiality under dynamic IIoT workloads.

A. Adaptive Resource Management in Edge–Fog Computing

Efficient workload allocation remains a fundamental challenge in heterogeneous edge–fog environments where computational capacity, communication latency, and energy availability vary significantly across nodes. Existing approaches can generally be categorized into optimization-based, learning-based, and distributed resource-management schemes.

Optimization-based methods employ heuristic or multi-criteria decision models to improve resource utilization and scheduling efficiency. Representative examples include FOCALB [11], multi-criteria resource-allocation frameworks [12], [13], and energy-aware scheduling mechanisms [10]. More recently, advanced optimization techniques such as memory-driven Gray Wolf Optimization (SIREN) [38] and mobility-aware resource-allocation frameworks [25] have been proposed to improve adaptation under dynamic edge-cloud environments. While these approaches achieve effective task placement, their optimization processes often incur substantial computational overhead and may struggle to respond rapidly to highly dynamic IIoT workloads.

Learning-based solutions utilize reinforcement learning, federated learning, and AI-assisted decision making to adapt scheduling policies according to changing operating conditions [14]–[16], [22]. Recent studies further explore intelligent network slicing and adaptive resource orchestration in multi-tier edge-cloud systems [17], [26]. Although these methods improve long-term scheduling performance, they frequently require extensive training data, retraining under workload

TABLE I: Comparison with representative fog and IIoT frameworks

Scheme	Adaptive Resource	Predictive Adaptation	Unified Recovery	Hier. Aggregation	Encrypted Aggregation	Secure Delegation	Fault-Loss Mitigation
FOCALB [11]	✓	✗	✗	✗	✗	✗	✗
ELECTRE [12]	✓	✗	✗	✗	✗	✗	✗
SDN-Based LB [15]	✓	P	✗	✗	✗	✗	✗
FedDQN [22]	✓	✓	✗	✗	✗	✗	✗
Replica Scheduling [19]	P	✗	P	✗	✗	✗	✗
Fault-Aware Scheduling [20]	P	✗	P	✗	✗	✗	✗
Paillier Aggregation [7]	✗	✗	✗	✗	✓	✗	✗
Robust IIoT Aggregation [24]	✗	✗	P	✗	✓	✗	P
P2-SWAN [33]	✗	✗	✗	✗	✓	P	✗
PRE-Based Delegation [27]	✗	✗	✗	✗	P	✓	✗
PLOSHA-RMFR (Ours)	✓	✓	✓	✓	✓	✓	✓

shifts, and additional computational resources, limiting their applicability in latency-sensitive industrial environments.

Distributed resource-management strategies attempt to eliminate centralized bottlenecks by exploiting decentralized information exchange mechanisms. Gossip-based dissemination and epidemic-style coordination protocols [30] provide scalable workload awareness without requiring global system knowledge. However, most distributed approaches focus primarily on workload balancing and do not explicitly account for failure risk or aggregation-loss exposure during scheduling decisions.

Consequently, existing resource-management solutions mainly optimize performance metrics such as latency, throughput, or energy consumption, while providing limited support for integrated resilience-aware workload adaptation.

B. Fault-Tolerant Recovery in Edge–Fog Systems

Fault tolerance is critical in IIoT deployments because node failures, communication disruptions, and workload surges can significantly degrade service availability. Existing fault-tolerance techniques primarily rely on replication, checkpointing, task migration, or recovery-based scheduling.

Several studies employ replication and redundancy mechanisms to improve service continuity [19], [35]. Although replication enhances reliability, it increases storage consumption, communication overhead, and energy usage. Checkpointing and workflow-recovery approaches have also been proposed to support task continuation under failures [37], [41]. However, these methods often introduce recovery latency and require maintaining additional execution-state information.

Recent research explores fault-aware scheduling and adaptive offloading mechanisms for edge environments. Examples include fault-tolerant task offloading using reinforcement learning [**?**], fault-aware workflow scheduling under fluctuating cloud resources [37], and preference-aware fault-tolerant embedding in serverless edge infrastructures [40]. EdgeHydra [42] further employs erasure coding to improve fault-tolerant edge data distribution. While these techniques improve resilience, they typically address failures only after they occur and treat resource allocation and fault recovery as separate optimization problems.

Moreover, most existing solutions focus on service continuity rather than minimizing the amount of partially processed data lost during failures. The relationship between workload allocation, aggregation structure, and recovery cost therefore remains largely unexplored.

C. Privacy-Preserving Aggregation and Secure Delegation

Protecting sensitive IIoT data during processing and transmission has motivated extensive research on privacy-preserving aggregation and secure delegation. Homomorphic encryption schemes, particularly those based on Paillier cryptography, enable aggregation directly over encrypted sensor readings without exposing plaintext data [7], [23], [24]. More recent systems extend encrypted aggregation toward secure analytics and large-scale data processing in fog and cloud environments [25], [26].

Despite these advances, most encrypted aggregation frameworks adopt fixed aggregation windows and flat aggregation structures. As a result, aggregation-loss exposure grows proportionally with aggregation-window size, and partial aggregation progress is often discarded when failures occur. Existing schemes therefore provide limited support for workload-aware aggregation adaptation or failure-aware aggregation preservation.

Secure workload delegation introduces an additional challenge because migrated tasks may require access to cryptographic keys or sensitive intermediate states. Proxy Re-Encryption (PRE) techniques have been widely investigated to support secure delegation across distributed environments [27]. However, PRE-based approaches require re-encryption key material to be available at intermediary nodes, thereby increasing trust assumptions and expanding the attack surface.

To reduce these risks, recent studies advocate hardware-assisted trust mechanisms based on TEEs, which provide isolated execution environments for sensitive computations and key management [28]. Nevertheless, existing TEE-assisted systems generally focus on secure execution and do not tightly integrate delegation decisions with adaptive recovery and aggregation processes.

Table I compares PLOSHA-RMFR with representative fog-computing frameworks. Existing resource-management solutions primarily focus on workload distribution and scheduling efficiency [11], [12], [15], [22], while fault-tolerance approaches emphasize replication or fault-aware scheduling [19], [20]. Privacy-preserving aggregation schemes support encrypted processing of sensor data [7], [24], [33], whereas secure delegation studies mainly address key-transfer and re-encryption mechanisms [27]. However, none of these ap-

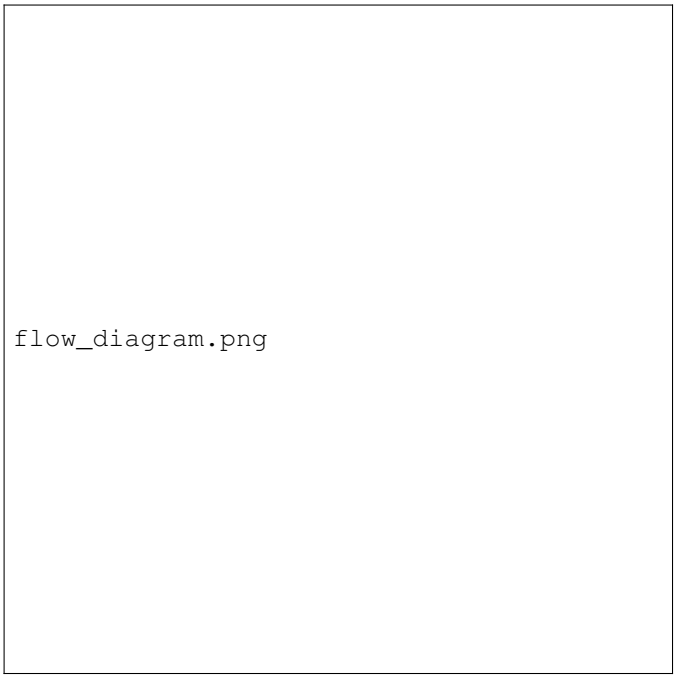

Fig. 1: PLOSHA-RMFR System Model.

proaches jointly provide predictive adaptation, unified recovery, hierarchical encrypted aggregation, secure delegation, and fault-loss-aware aggregation within a single framework. In contrast, PLOSHA-RMFR integrates adaptive hierarchical aggregation, risk-aware multi-layer fault recovery, and TEE-protected delegation to simultaneously improve scalability, resilience, and confidentiality in IIoT edge–fog environments.

III. Our Proposed PLOSHA-RMFR Scheme

This section presents the proposed **PLOSHA-RMFR** (**P**redictive **A**daptive **H**ierarchical **S**lot **A**ggregation with **R**isk-**A**ware **M**ulti-**L**ayer **F**ault **R**ecovery) framework for secure and resilient IIoT edge–fog computing. PLOSHA-RMFR jointly addresses three fundamental challenges in fog-assisted IIoT environments: efficient encrypted aggregation, proactive resilience against overloads and failures, and secure workload delegation. The framework integrates Predictive Adaptive Hierarchical Slot Aggregation (PLOSHA), Risk-Aware Multi-Layer Fault Recovery (RMFR), and TEE-protected fog-scoped delegation within a unified architecture to improve scalability, fault resilience, and confidentiality while minimizing aggregation-loss exposure.

A. System Model

Fig. 1 illustrates the system architecture of the proposed PLOSHA-RMFR framework. The framework comprises four entities: Industrial IoT Sensors ($\mathcal{S}$), Fog Nodes ($\mathcal{F}$), a Key and Reliability Management module (KRM), and a Cloud Server ($\mathcal{C}$). Sensors continuously generate operational measurements and transmit encrypted readings to nearby fog nodes. Each fog node executes the proposed PLOSHA mechanism to perform adaptive micro-slot aggregation over encrypted data while monitoring workload, queue utilization, latency, and reliability conditions. The KRM manages key provisioning, reliability-state maintenance, remote attestation, and secure delegation among fog nodes. Aggregated ciphertexts are subsequently forwarded to the cloud for long-term storage and analytics without exposing plaintext sensor data.

Industrial IoT Sensors ($\mathcal{S}$): A set of resource-constrained sensors continuously generate operational data such as vibration, temperature, pressure, and equipment-health measurements. Before transmission, each reading is encrypted and forwarded to its associated fog node for aggregation and processing.

Fog Nodes ($\mathcal{F}$): Fog nodes are deployed close to data sources and serve as the primary processing layer of the framework. Each fog node executes the proposed PLOSHA mechanism, which dynamically partitions aggregation epochs into adaptive micro-slots according to estimated workload and reliability conditions. Fog nodes perform encrypted aggregation without accessing plaintext sensor data and maintain local workload statistics, queue information, reliability scores, and neighborhood status information.

Key and Reliability Management Module (KRM): The KRM is a trusted management entity responsible for key provisioning, remote attestation, reliability-state management, and delegation authorization. The KRM provisions fog-scoped cryptographic credentials exclusively to attested TEE enclaves and coordinates secure workload migration during overload mitigation and recovery operations.

Cloud Server ($\mathcal{C}$): The cloud server provides long-term storage, archival services, and analytical processing over aggregated data. The cloud is considered honest-but-curious and receives only encrypted aggregate outputs generated by fog nodes.

The framework operates in discrete aggregation epochs. During each epoch, PLOSHA adaptively determines the aggregation hierarchy and micro-slot granularity based on estimated workload and reliability conditions obtained through lightweight EWMA-based prediction. When overloads, node degradation, or communication failures are anticipated, RMFR proactively initiates workload redistribution, ACK-assisted recovery, or gossip-assisted failover while preserving aggregation correctness and minimizing aggregation-loss exposure.

B. Threat Model

PLOSHA-RMFR assumes a semi-trusted IIoT edge–fog environment consisting of sensors, fog nodes, cloud services, and the KRM authority. Adversaries are modeled as probabilistic polynomial-time (PPT) entities capable of eavesdropping on communication channels, compromising fog hosts, injecting malicious traffic, replaying messages, and attempting to disrupt aggregation and recovery operations.

The cloud server is considered *honest-but-curious*. It correctly stores aggregated ciphertexts and executes requested operations but may attempt to infer sensitive information from received data. Similarly, compromised fog hosts may inspect local memory, operating-system state, or communication traffic. However, Trusted Execution Environments are assumed secure, and cryptographic keys stored inside attested enclaves cannot be extracted by adversaries.

The adversary may attempt to:

1) infer sensor readings from intercepted communications;
2) compromise a fog node and access aggregation data;
3) exploit overload conditions to degrade service availability;
4) trigger repeated recovery operations to increase system overhead;
5) manipulate workload-distribution decisions through falsified status information;
6) disrupt aggregation by causing node failures during active aggregation epochs.

PLOSHA-RMFR does not assume trusted communication channels between sensors, fog nodes, and the cloud. Standard cryptographic assumptions underlying AES and Paillier encryption are assumed to hold. Furthermore, the adversary cannot simultaneously compromise the KRM and all participating TEE enclaves.

The primary security objectives of the framework are:

- **Confidentiality:** Sensor readings remain protected during transmission, aggregation, delegation, and storage.
- **Compartmentalized Key Exposure:** Compromise of a fog node affects only the corresponding fog-scoped key domain and does not expose system-wide cryptographic material.
- **Aggregation Correctness:** Encrypted aggregation results remain correct under workload redistribution, delegation, and recovery operations.
- **Availability and Resilience:** The framework continues processing sensor workloads despite overload conditions, communication failures, and fog-node disruptions.
- **Fault-Loss Minimization:** Aggregation-loss exposure is restricted to affected micro-slots rather than entire aggregation epochs.

C. System Process

The PLOSHA-RMFR framework operates through five sequential phases where its details are described below. Table II summarizes the major notations used throughout the framework.

Phase I: System Initialization and Secure Provisioning: This phase establishes the trust relationships, cryptographic credentials, and operational states required by the PLOSHA-RMFR framework.

Step 1: Entity Registration and Attestation. Let $\mathcal{S} = S_1, S_2, \ldots, S_m$ and $\mathcal{F} = F_1, F_2, \ldots, F_n$ denote the sets of IIoT sensors and fog nodes, respectively. Before participating in the framework, each fog node $F_i \in \mathcal{F}$ performs remote attestation with the Key and Reliability Management module (KRM). A fog node is admitted only if $Attest(F_i) = 1$. Successful attestation establishes a Trusted TEE, which serves as the secure execution platform for aggregation, delegation, and recovery operations.

Step 2: Sensor Association and Key Provisioning. The KRM initializes a sensor-to-fog assignment function $\Gamma : \mathcal{S} \to \mathcal{F}$, where $\Gamma(S_j) = F_i$ indicates that sensor S_j is assigned to fog node F_i. For each fog node F_i, the KRM generates a unique fog-scoped AES-GCM key k_i and securely provisions it to the enclave of F_i and all sensors satisfying $\Gamma(S_j) = F_i$. Consequently, each fog node maintains an independent cryptographic domain, thereby localizing the impact of key compromise to a single fog region and preventing system-wide key exposure.

TABLE II: Major Notations Used in PLOSHA-RMFR

Notation	Description
$\mathcal{S}$	Set of IIoT sensors
$\mathcal{F}$	Set of fog nodes
$\mathcal{C}$	Cloud server
KRM	Key and Reliability Management module
F_i	Fog node i
S_j	Sensor j
k_i	Fog-scoped AES key of F_i
(pk_P, sk_P)	Paillier public/private key pair
$W_i(t)$	Current workload of F_i
$\hat{W}_i(t)$	Predicted workload of F_i
$Rel_i(t)$	Reliability score of F_i
$Q_i(t)$	Queue utilization of F_i
$L_i(t)$	Communication latency of F_i
λ_i	Estimated failure probability of F_i
$Risk_i$	RMFR risk score
τ	Overload threshold
Δ	Aggregation epoch
m	Number of adaptive micro-slots
m^*	Optimal micro-slot number
δ_k	Micro-slot k
C_j	Paillier ciphertext of reading j
$C_{micro,k}$	Micro-slot aggregate ciphertext
$C_{agg,i}$	Fog aggregate ciphertext
α, β	EWMA weighting factors

Step 3: Homomorphic Aggregation Setup. To support privacy-preserving aggregation, the KRM generates a Paillier key pair (pk_P, sk_P) using security parameter λ. The public key pk_P is provisioned to all attested fog enclaves to enable homomorphic aggregation, whereas the private key sk_P remains sealed within the KRM and is never disclosed to fog nodes. As a result, fog nodes can aggregate encrypted sensor readings without obtaining decryption capability.

Step 4: Reliability and Operational-State Initialization. For each fog node F_i, the KRM initializes the reliability score as

$[\ \mathrm{Rel}_i(0) = 1,]$

indicating a fully operational state at deployment time. The corresponding initial operational state is represented by

$[\ \mathrm{State}_i(0)$

$\big[W_i(0)\ Q_i(0)\ L_i(0)\ Rel_i(0)\big].\ ]$

Here, $W_i(0)$, $Q_i(0)$, and $L_i(0)$ denote the initial workload, queue utilization, and communication latency, respectively, while $Rel_i(0)$ represents the initial reliability score. These values establish the baseline operational profile of fog node F_i and serve as the reference state for predictive estimation, capacity evaluation, and risk analysis in subsequent phases.

Step 5: Initialization Output. The initialization procedure outputs the system state

$$\mathcal{O}_{init} = \{pk_P,\ \Gamma,\ \{k_i\}_{i=1}^n,\ \{Rel_i(0)\}_{i=1}^n,\ \{State_i(0)\}_{i=1}^n\}$$

which forms the operational foundation for predictive workload estimation, adaptive hierarchical aggregation, and risk-aware fault recovery throughout the PLOSHA-RMFR framework.

Phase II: Predictive Capacity and Risk Estimation: This phase proactively predicts the future operating condition of each fog node to support adaptive aggregation planning and risk-aware recovery. Rather than reacting after overload or service degradation occurs, PLOSHA-RMFR continuously estimates workload evolution, queue congestion, communication latency, and reliability trends using an Exponential Weighted Moving Average (EWMA) predictor. The resulting prediction enables proactive aggregation adaptation and early fault mitigation before service quality deteriorates.

Step 1: Runtime State Collection.

During each aggregation epoch Δ, every fog node F_i continuously monitors its operational condition and periodically reports a runtime state vector to the Key and Reliability Management (KRM) module:

$$[\ \text{State}_i(t)\ \big[W_i(t)\ Q_i(t)\ L_i(t)\ Rel_i(t)\big]\ .]$$

Here, $W_i(t)$, $Q_i(t)$, $L_i(t)$, and $Rel_i(t)$ denote the normalized workload, queue utilization, communication latency, and reliability score of fog node F_i, respectively. All state variables are normalized to the interval $[0, 1]$.

Step 2: Future State Prediction.

To smooth short-term fluctuations while preserving responsiveness to dynamic operating conditions, PLOSHA-RMFR employs an EWMA predictor:

$$[\ \widehat{State}_i(t+1)\alpha State_i(t) + (1-\alpha)\widehat{State}_i(t),\ ]$$

where $\alpha \in (0, 1)$ denotes the smoothing coefficient.

Consequently,

$$[\ \widehat{State}_i(t+1)\ \Big[\widehat{W}_i(t+1)\ \widehat{Q}_i(t+1)\ \widehat{L}_i(t+1)\ \widehat{Rel}_i(t+1)\Big]\ .]$$

The predicted state vector provides a short-term estimate of the future operating condition of fog node F_i.

Step 3: Effective Aggregation Capacity Estimation.

Using the predicted state, PLOSHA-RMFR evaluates the future aggregation capability of fog node F_i as

$$[\ \text{Cap}_i(t+1)\widehat{Rel}_i(t+1)\Big[1-\big(\omega_w\widehat{W}_i(t+1)+\omega_q\widehat{Q}_i(t+1)+\omega_l\widehat{L}_i(t+1)\big)\Big],\ ]$$

subject to

$$[\ \omega_w+\omega_q+\omega_l=1, \qquad \omega_w,\omega_q,\omega_l\in[0,1].]$$

Since all variables are normalized,

$$[\ 0\leq Cap_i(t+1)\leq 1.]$$

A larger value indicates greater available processing resources, lower congestion, and stronger operational stability.

Step 4: Failure Exposure Analysis.

To estimate the likelihood of aggregation disruption, PLOSHA-RMFR computes the failure exposure score

$$[\ \text{FE}_i(t)\widehat{W}_i(t+1)\widehat{Q}_i(t+1)\Big(1-\widehat{Rel}_i(t+1)\Big).]$$

The resulting score satisfies

$$[\ 0\leq FE_i(t)\leq 1.]$$

A larger value indicates increased vulnerability to aggregation interruption caused by simultaneous workload pressure, queue congestion, and reliability degradation.

Step 5: Operational Risk Estimation.

The overall operational risk is determined by jointly considering capacity degradation and failure exposure:

$$[\ \text{Risk}_i(t)\eta_1\Big(1-Cap_i(t+1)\Big)+\eta_2 FE_i(t),\ ]$$

subject to

$$[\ \eta_1+\eta_2=1, \qquad \eta_1,\eta_2\in[0,1].]$$

Consequently,

$$[\ 0\leq Risk_i(t)\leq 1.]$$

A larger value indicates that the fog node is approaching overload, instability, or service degradation.

Step 6: Risk Classification.

Based on the estimated operational risk, the framework categorizes the operating condition of fog node F_i as

$$[\ \text{Status}_i(t)\begin{cases}\text{Stable}, & Risk_i(t)<\tau_r,\ [1mm]\text{Critical}, & Risk_i(t)\geq\tau_r.\end{cases}]$$

where τ_r denotes the adaptive risk threshold maintained by AFLTO.

Step 7: Prediction Output.

Finally, the prediction phase generates the prediction state vector

$$[\ \text{Pred}_i(t)\ \big[Cap_i(t+1)\ FE_i(t)\ Risk_i(t)\big]\ .]$$

The prediction vector is forwarded to PLOSHA for adaptive aggregation planning and to RMFR for risk-aware recovery decision making.

Phase III: Predictive Adaptive Hierarchical Slot Aggregation (PLOSHA): This phase performs privacy-preserving aggregation of sensor readings while dynamically adapting the aggregation structure according to the predicted operating condition of each fog node. Unlike conventional aggregation schemes that employ fixed aggregation windows, PLOSHA continuously adjusts aggregation granularity according to predicted aggregation capacity, failure exposure, and reliability conditions. Consequently, the framework reduces aggregation latency, minimizes aggregation-loss exposure, and improves resilience against overloads and fog-node failures.

Step 1: Prediction-Driven Aggregation Planning.

For each fog node F_i, PLOSHA receives the prediction vector generated in Phase II:

$$[\ \text{Pred}_i(t)\ \big[Cap_i(t+1)\ FE_i(t)\ Risk_i(t)\big]\ .]$$

The prediction vector summarizes the anticipated aggregation capacity, failure exposure, and operational risk of fog node F_i.

Let m denote the number of micro-slots within aggregation epoch Δ, where

$$[\ 1\leq m\leq m_{\max}.]$$

To model aggregation overhead, PLOSHA defines

$$[\ \text{T}_{\text{agg}}(m)\beta_t m,\ ]$$

where β_t denotes the average processing overhead associated with a single micro-slot.

To quantify aggregation-loss exposure, PLOSHA defines

$$[\ \text{L}_{\text{agg}}(m)\frac{|\delta_k|}{\sum_{r=1}^{m}|\delta_r|}.]$$

Under uniform epoch partitioning,

$$[\ \text{L}_{\text{agg}}(m)\frac{1}{m}.]$$

This quantity represents the maximum fraction of the aggregation epoch that may be lost when a single micro-slot becomes unavailable.

The optimal aggregation granularity is selected as

$$[\ \text{m}^*\arg\min_{1\leq m\leq m_{\max}}\Big[\lambda_1 T_{\text{agg}}(m)\Big(1-Cap_i(t+1)\Big)+\lambda_2 FE_i(t)L_{\text{agg}}(m)+\lambda_3\Big(1-Rel_i(t)\Big)L_{\text{agg}}(m)\Big],\ ]$$

subject to

$$[\ \lambda_1+\lambda_2+\lambda_3=1, \qquad \lambda_1,\lambda_2,\lambda_3\in[0,1].]$$

The optimization jointly balances aggregation efficiency, failure exposure, and reliability preservation. Consequently, larger failure exposure or lower reliability naturally increases the number of micro-slots, thereby improving fault isolation and reducing aggregation-loss exposure.

Step 2: Adaptive Epoch Partitioning.

Based on the optimized value m^*, the aggregation epoch is partitioned into the micro-slot set

[$\mathrm{D}_i \delta_k{}_{k=1}^{m^*}$.]

Each micro-slot δ_k represents a temporal aggregation interval containing sensor reports received during that interval.

Under stable operating conditions, PLOSHA selects fewer micro-slots to reduce aggregation overhead and processing latency. Conversely, under adverse operating conditions, additional micro-slots are introduced to improve fault localization and recovery efficiency.

Step 3: Secure Data Collection and Ciphertext Transformation.

For each sensor S_j associated with fog node F_i, the sensed value d_j is encrypted using the fog-scoped AES-GCM key k_i:

[$\mathrm{CT}_j Enc_{k_i}(d_j)$.]

Direct Paillier encryption at resource-constrained sensors incurs substantial computational, memory, and energy overhead. Therefore, PLOSHA performs lightweight AES-GCM encryption at the sensor layer and securely outsources ciphertext transformation to the attested TEE enclave of the fog node.

The TEE enclave serves as a trusted aggregation boundary that transforms lightweight ciphertexts into homomorphic ciphertexts without exposing plaintext values outside the enclave.

Upon reception, the ciphertext is processed exclusively within the enclave:

[$\mathrm{d}_j Dec_{k_i}(CT_j),$]

[$\mathrm{C}_j Enc_{pk_P}(d_j)$.]

The plaintext value exists only transiently within the protected enclave and is immediately erased following transformation.

Compared with direct Paillier encryption at the sensor layer, this approach significantly reduces sensor-side computational complexity, memory consumption, and energy expenditure. Furthermore, Paillier key material remains confined to trusted infrastructure, thereby reducing cryptographic exposure and simplifying key management across large-scale IIoT deployments.

Step 4: Micro-Slot Aggregation.

For each micro-slot $\delta_k \in \mathcal{D}_i$, the enclave performs homomorphic aggregation over all Paillier ciphertexts associated with that interval:

[$\mathrm{C}_{\mathrm{micro},k} \prod_{j \in \delta_k} C_j \pmod{n_P^2}$.]

By the additive homomorphic property of Paillier encryption,

[$\mathrm{Dec}_{sk_P}!\left(C_{\mathrm{micro},k}\right) \sum_{j \in \delta_k} d_j$.]

Step 5: Hierarchical Fog Aggregation.

The encrypted micro-slot aggregates are recursively combined to produce the fog-level aggregate:

[$\mathrm{C}_{\mathrm{agg},i} \prod_{k=1}^{m^*} C_{\mathrm{micro},k} \pmod{n_P^2}$.]

Consequently,

[$\mathrm{Dec}_{sk_P}!\left(C_{\mathrm{agg},i}\right) \sum_{k=1}^{m^*} \sum_{j \in \delta_k} d_j$.]

The hierarchical aggregation structure localizes aggregation loss to individual micro-slots. Therefore, failures affecting a specific micro-slot require recovery only for that micro-slot rather than the entire aggregation epoch.

Step 6: Aggregation Completeness Assessment.

To identify incomplete aggregation caused by communication failures, sensor outages, or fog-node instability, PLOSHA computes the aggregation completeness score

[$\mathrm{V}_i(t) \frac{N_{\mathrm{recv}}(t)}{N_{\mathrm{exp}}(t)}$.]

The expected number of reports is determined from the active sensor registry maintained by the KRM:

[$\mathrm{N}_{\mathrm{exp}}(t) \sum_{S_j \in \Gamma_i(t)} Act_j(t),$]

where

[$\mathrm{Act}_j(t) \begin{cases} 1, & \text{sensor } S_j \text{ is active,} \\ \text{otherwise.} \end{cases}$ [2mm]0,]

Consequently, the completeness metric remains valid under dynamic sensor participation, temporary disconnections, and varying IIoT workloads.

The aggregation completeness indicator is defined as

[$\Phi_i(t) \begin{cases} 1, & V_i(t) < \tau_v, \\ \text{otherwise.} \end{cases}$ [2mm]0,]

where τ_v denotes the adaptive completeness threshold optimized by AFLTO.

Step 7: Aggregation Output.

Finally, PLOSHA generates the aggregation state tuple

[$\mathrm{Agg}_i(t)\Big(C_{\mathrm{agg},i}, m^*, V_i(t), \Phi_i(t), Cap_i(t+1), Risk_i(t), Rel_i(t)\Big)$.]

The aggregation state tuple summarizes the encrypted aggregate, adaptive aggregation configuration, aggregation completeness status, predicted aggregation capacity, operational risk, and current reliability score.

The resulting tuple is forwarded to the Risk-Aware Multi-Layer Fault Recovery (RMFR) module for recovery decision making.

Aggregation-Loss Localization Property.

Because PLOSHA partitions each aggregation epoch into m^* micro-slots, a failure affecting a single micro-slot impacts at most

[$\mathrm{L}_{\mathrm{agg}}(m^*) \frac{1}{m^*}$]

of the aggregation epoch under uniform partitioning.

Consequently, aggregation-loss exposure decreases monotonically as the number of micro-slots increases, thereby providing the foundation for localized recovery in RMFR and adaptive threshold optimization in AFLTO.

Phase IV: Risk-Aware Multi-Layer Fault Recovery (RMFR): This phase preserves aggregation continuity under overload, incomplete aggregation, communication disruption, and fog-node failure. Unlike conventional fault-tolerance mechanisms that rely on reactive failover or full replication,

RMFR adopts a progressive recovery-escalation strategy that selects the least costly recovery action capable of restoring aggregation correctness and service continuity. By exploiting the hierarchical aggregation structure established in Phase III, RMFR localizes recovery to affected micro-slots whenever possible, thereby reducing recovery overhead, communication cost, and aggregation-loss exposure.

Step 1: Recovery Urgency Evaluation.

For each fog node F_i, RMFR receives the aggregation state tuple

$$Agg_i(t) = \Big(C_{\mathrm{agg},i}, m^*, V_i(t), \Phi_i(t), Cap_i(t{+}1), Risk_i(t), Rel_i(t)\Big).$$

RMFR computes the recovery urgency score as

$$RU_i(t) = \rho_1 Risk_i(t) + \rho_2\big(1 - V_i(t)\big) + \rho_3\big(1 - Rel_i(t)\big),$$

where

$$\rho_1 + \rho_2 + \rho_3 = 1, \qquad \rho_1, \rho_2, \rho_3 \in [0, 1].$$

Since all variables are normalized,

$$0 \le RU_i(t) \le 1.$$

A larger value of $RU_i(t)$ indicates a higher probability of aggregation disruption and service degradation.

Step 2: Recovery Escalation Decision.

RMFR first checks the aggregation completeness indicator $\Phi_i(t)$. If $\Phi_i(t) = 1$, recovery is mandatory. The recovery mode is determined as

$$Mode_i(t) = \begin{cases} \mathsf{Normal}, & \Phi_i(t) = 0 \wedge RU_i(t) < \tau_1, \\ \mathsf{Delegation}, & \Phi_i(t) = 0 \wedge \tau_1 \le RU_i(t) < \tau_2, \\ \mathsf{MicroRecovery}, & \Phi_i(t) = 1 \wedge RU_i(t) < \tau_3, \\ \mathsf{Failover}, & RU_i(t) \ge \tau_3 \vee Rel_i(t) \le \tau_f. \end{cases}$$

where

$$0 \le \tau_1 < \tau_2 < \tau_3 \le 1, \qquad \tau_f \in [0, 1].$$

This escalation strategy ensures that incomplete aggregation is recovered before cloud commitment and that critically unreliable fog nodes are proactively removed from service.

Step 3: Recovery Candidate Selection.

When $Mode_i(t) \in \{\mathsf{Delegation}, \mathsf{Failover}\}$, RMFR evaluates each neighboring fog node $F_j \in \mathcal{N}_i$ using

$$U_j(t) = \alpha_c Cap_j(t+1) + \alpha_r Rel_j(t) + \alpha_k\big(1 - Risk_j(t)\big),$$

where

$$\alpha_c + \alpha_r + \alpha_k = 1, \qquad \alpha_c, \alpha_r, \alpha_k \in [0, 1].$$

The optimal recovery candidate is selected as

$$F_i^* = \arg\max_{F_j \in \mathcal{N}_i} U_j(t).$$

A larger utility indicates stronger recovery suitability, higher reliability, and lower operational risk.

Step 4: Layer-I Predictive Delegation Recovery.

When $Mode_i(t) = \mathsf{Delegation}$, the current aggregation process remains complete, but future overload or instability is predicted. RMFR constructs the delegation state package

$$DSP_i(t) = \Big(m^*, C_{\mathrm{agg},i}, Seq_i(t), Cap_i(t{+}1), Risk_i(t), Rel_i(t)\Big),$$

where $Seq_i(t)$ denotes the aggregation sequence number of the current epoch. The KRM securely seals and transmits $DSP_i(t)$ to the attested TEE enclave of F_i^*. The delegated fog node then performs shadow aggregation in subsequent epochs and prepares to absorb future workload without immediate sensor reassociation. This proactive delegation mitigates overload before aggregation quality deteriorates.

Step 5: Layer-II Selective Micro-Slot Recovery.

When $Mode_i(t) = \mathsf{MicroRecovery}$, the fog node remains operational but aggregation incompleteness is detected. RMFR identifies the incomplete micro-slot set as

$$\mathcal{D}_i^{miss} = \left\{\delta_k \in \mathcal{D}_i \ : \ N_{\delta_k}^{recv} < N_{\delta_k}^{exp}\right\},$$

and the valid micro-slot set as

$$\mathcal{D}_i^{valid} = \mathcal{D}_i \setminus \mathcal{D}_i^{miss}.$$

For each incomplete micro-slot, missing ciphertext reports are retransmitted from sensors or reconstructed from locally buffered ciphertexts maintained during the current aggregation epoch. The recovered micro-slot aggregate is computed as

$$C_{\mathrm{micro},k}^{rec} = \prod_{j \in \delta_k^{rec}} C_j.$$

The recovered fog-level aggregate becomes

$$C_{\mathrm{agg},i}^{rec} = \left(\prod_{\delta_k \in \mathcal{D}_i^{valid}} C_{\mathrm{micro},k}\right)\left(\prod_{\delta_k \in \mathcal{D}_i^{miss}} C_{\mathrm{micro},k}^{rec}\right).$$

Since only incomplete micro-slots are recomputed, RMFR avoids full-epoch reaggregation and reduces recovery latency and communication overhead.

Step 6: Layer-III Reliability-Aware Failover Recovery.

When $Mode_i(t) = \mathsf{Failover}$, the current fog node is considered unavailable or critically unstable. The replacement fog node is selected as

$$F_i^* = \arg\max_{F_j \in \mathcal{N}_i} U_j(t).$$

The KRM provisions a temporary recovery credential to the attested TEE enclave of F_i^* and migrates the failover state

$$FSM_i(t) = \Big(m^*, C_{\mathrm{agg},i}, \mathcal{D}_i, \Phi_i(t), Rel_i(t)\Big).$$

The sensor-association mapping is then updated as

$$\Gamma(S_j) \leftarrow F_i^*, \qquad \forall S_j : \Gamma(S_j) = F_i.$$

Consequently, sensors previously associated with F_i are redirected to the replacement fog node in subsequent aggregation epochs.

Step 7: Reliability Reinforcement and Recovery Output. The recovery-success indicator is defined as

$$Succ_i(t) = \begin{cases} 1, & \text{aggregation successfully completed,} \\ 0, & \text{otherwise.} \end{cases}$$

The reliability score is updated as

$$Rel_i(t{+}1) = \min\Big\{1,\ \beta_r Rel_i(t) + (1-\beta_r)\Big[\lambda_s Succ_i(t) + \lambda_v V_i(t)\Big]\Big\},$$

where

$$\lambda_s + \lambda_v = 1, \qquad \lambda_s, \lambda_v \in [0,1].$$

This update rewards successful recovery and aggregation completeness while preserving long-term reliability history. The recovery status is defined as

$$RecStatus_i(t) = Succ_i(t).$$

Finally, RMFR generates the recovery state tuple

$$Rec_i(t) = \Big(Mode_i(t), RU_i(t), F_i^*, Rel_i(t{+}1), RecStatus_i(t)\Big).$$

The recovery state tuple is forwarded to AFLTO in Phase V for adaptive threshold optimization and closed-loop system refinement.

Recovery Complexity Analysis.

Because RMFR localizes recovery to affected micro-slots only, the recovery complexity is proportional to the number of incomplete micro-slots:

$$O\big(|\mathcal{D}_i^{miss}|\big),$$

rather than the total number of micro-slots,

$$O(m^*).$$

Consequently, recovery overhead remains bounded even under large aggregation epochs, thereby improving scalability and fault resilience in large-scale IIoT deployments.

To summarize the end-to-end operation of the proposed framework, Algorithm 1 presents the overall PLOSHA-RMFR aggregation and recovery procedure, including predictive workload assessment, adaptive aggregation, encrypted processing, and multi-layer fault recovery.

Phase V: Adaptive Feedback Learning and Threshold Optimization (AFLTO): This phase securely commits the final aggregation result and continuously refines the operational parameters of PLOSHA-RMFR. Unlike conventional aggregation frameworks that terminate after recovery, AFLTO establishes a closed-loop optimization process that continuously adapts aggregation and recovery behavior according to observed aggregation quality, recovery pressure, and reliability conditions.

Step 1: Final Aggregate Commitment.

Algorithm 1 PLOSHA-RMFR Aggregation and Recovery Procedure

Require: $State_i(t)$, Δ, $\mathcal{S}_i$, $\mathcal{N}_i$
Ensure: $Rec_i(t)$

```
Compute State_i(t+1)
Derive Cap_i(t+1), FE_i(t), Risk_i(t)
Determine optimal micro-slot number m*
Generate D_i = {δ_1, δ_2, ..., δ_{m*}}
for all CT_j ∈ S_i do
    C_j ← TEETransform(CT_j)
end for
for all δ_k ∈ D_i do
    C_{micro,k} ← ∏_{j∈δ_k} C_j
end for
C_{agg,i} ← ∏_{k=1}^{m*} C_{micro,k}
Compute V_i(t)
Compute RU_i(t)
Determine Mode_i(t)
if Mode_i(t) = Delegation then
    F_i* ← arg max_{F_j∈N_i} U_j(t)
    Delegate DSP_i(t) to F_i*
else if Mode_i(t) = MicroRecovery then
    Determine D_i^{miss}
    Reconstruct C_{agg,i}^{rec}
else if Mode_i(t) = Failover then
    F_i* ← arg max_{F_j∈N_i} U_j(t)
    Migrate FSM_i(t) to F_i*
end if
Generate Rec_i(t)
return Rec_i(t)
```

AFLTO first receives the recovery state

[$\text{Rec}_i(t)\Big(Mode_i(t), RU_i(t), F_i^*, Rel_i(t+1), RecStatus_i(t)\Big)$.]

The final aggregate is determined as

$$C_i^{\text{final}} = \begin{cases} C_{\text{agg},i}, & \Phi_i(t) = 0, \\ C_{\text{agg},i}^{\text{rec}}, & \Phi_i(t) = 1, \\ & RecStatus_i(t) = 1, \\ C_{\text{agg},i}, & \Phi_i(t) = 1, \\ & RecStatus_i(t) = 0. \end{cases}$$

To ensure integrity and authenticity, the enclave constructs

[$\text{T}_i(t)\Big(C_i^{\text{final}}, Mode_i(t), Rel_i(t+1), RecStatus_i(t)\Big)$]

and generates

[$\sigma_i(t)\text{Sign}_{sk_i^{TEE}}\Big(H(T_i(t))\Big)$.]

The cloud stores

[$\Big(T_i(t), \sigma_i(t)\Big)$]

only after successful signature verification.

Step 2: Performance Evaluation.

Following cloud commitment, AFLTO evaluates the operational effectiveness of the current aggregation epoch. The evaluation jointly considers aggregation completeness and fog-node reliability to quantify the overall aggregation quality.

The aggregation quality score is computed as

$$Score_i(t) = \omega_1 V_i(t) + \omega_2 Rel_i(t+1),$$

where

$$\omega_1 + \omega_2 = 1, \qquad \omega_1, \omega_2 \in [0,1].$$

Since both $V_i(t)$ and $Rel_i(t+1)$ are normalized to the interval $[0,1]$,

$$0 \le Score_i(t) \le 1.$$

A larger value of $Score_i(t)$ indicates that the aggregation process achieved higher completeness while maintaining stronger operational reliability. This score serves as the primary performance indicator for the subsequent learning and threshold-optimization processes.

Step 3: Historical Learning and Error Estimation.

To capture long-term system behavior, AFLTO updates the historical performance profile [$\text{Hist}_i(t+1)\gamma Hist_i(t) + (1-\gamma)Score_i(t),]where[\gamma \in (0,1).]$

The current and historical observations are fused as [$\text{Score}_i^*(t)\alpha_h Hist_i(t+1) + (1-\alpha_h)Score_i(t),]$

where

[$\alpha_h \in (0,1).]$

The adaptive control error is then computed as

[$\text{e}_i(t)\kappa_1\Big(\bar{S}Score_i^*(t)\Big) + \kappa_2 RU_i(t) + \kappa_3\Big(1 - Rel_i(t+1)\Big),]$

subject to

[$\kappa_1 + \kappa_2 + \kappa_3 = 1.]$

A larger error indicates deteriorating aggregation quality, increasing recovery pressure, or reliability degradation.

Step 4: Adaptive Threshold Optimization.

Using the adaptive control error, AFLTO updates the aggregation and recovery thresholds through bounded optimization:

[$\tau_x(t+1)\Pi_{[0,1]}\Big(\tau_x(t) + \mu_x e_i(t)\Big),]$

where

[$\text{x} \in v, r, 1, 2, 3, f,]$

and

[$\Pi_{[0,1]}(y)\min 1, \max(0, y).]$

The ordering constraint

[$\tau_1(t+1) < \tau_2(t+1) < \tau_3(t+1)]$

is enforced after each update cycle to preserve recovery-escalation consistency.

As the adaptive error increases, the thresholds become more sensitive, enabling earlier intervention and stronger resilience. Conversely, when operational conditions improve, the thresholds gradually relax to reduce unnecessary recovery actions.

Step 5: Feedback Generation and Closed-Loop Adaptation.

Finally, AFLTO generates the feedback state

[$\text{FB}_i(t)\Big(Score_i(t), Hist_i(t+1), e_i(t), \tau_v(t+1), \tau_r(t+1), \tau_1(t+1), \tau_2(t+1), \tau_3(t+1), \tau_f(t+1)\Big).]$

The feedback state is supplied to the prediction and recovery modules during the next aggregation epoch:

[$\text{State}_i(t) \to \widehat{State}_i(t+1) \to Pred_i(t) \to Agg_i(t) \to Rec_i(t) \to FB_i(t).]$

Consequently, PLOSHA-RMFR continuously adapts its aggregation and recovery behavior according to evolving workload dynamics, reliability conditions, and recovery outcomes.

Adaptive Stability Property.

Since all threshold updates are bounded by the projection operator ($\Pi_{[0,1]}(\cdot)$),

[$0 \le \tau_v, \tau_r, \tau_1, \tau_2, \tau_3, \tau_f \le 1,]$

ensuring stable and bounded adaptation throughout long-term system operation.

IV. Security Analysis

A. Security Assumptions

The security of PLOSHA-RMFR relies on the following assumptions:

- AES-GCM provides IND-CPA confidentiality and ciphertext integrity.
- Paillier encryption is semantically secure under the Decisional Composite Residuosity Assumption (DCRA).
- Attested TEE enclaves provide isolated execution and protected key storage.
- The KRM securely provisions keys and authorizes delegation only to attested fog nodes.
- The adversary cannot simultaneously compromise the KRM and all participating TEE enclaves.

B. Theorem 1: End-to-End Confidentiality

Theorem 1 (End-to-End Confidentiality)**.** *Assume that:*

1) *AES-GCM provides IND-CPA confidentiality;*
2) *Paillier encryption is semantically secure under the Decisional Composite Residuosity Assumption (DCRA);*
3) *Trusted Execution Environments (TEEs) provide secure execution and memory isolation.*

Then no probabilistic polynomial-time (PPT) adversary can learn any information about sensor readings beyond negligible probability during transmission, aggregation, delegation, or cloud storage.

Proof. For each sensor S_j, the sensed value d_j is encrypted using the fog-scoped AES-GCM key k_i before transmission:

$$CT_j \mathsf{Enc}_{k_i}(d_j). \tag{1}$$

Consequently, an adversary observing the communication channel obtains only ciphertexts and cannot distinguish between two chosen plaintexts with non-negligible advantage under the IND-CPA security of AES-GCM.

After reception at fog node F_i, ciphertext transformation is performed exclusively inside an attested TEE enclave:

$$d_j \mathsf{Dec}_{k_i}(CT_j), \qquad C_j \mathsf{Enc}_{pk_P}(d_j). \tag{2}$$

The plaintext exists only transiently within the protected enclave and is immediately erased after transformation. Therefore, compromise of the host operating system does not reveal sensor values unless the TEE security assumption is violated.

The transformed ciphertext C_j is protected by Paillier encryption and remains encrypted throughout aggregation, delegation, recovery, and cloud storage. Since the Paillier private key sk_P is never disclosed outside the KRM, an adversary cannot recover plaintext values from aggregated ciphertexts.

Consider the following sequence of hybrid games:

- **Game** G_0**:** Real execution of PLOSHA-RMFR.
- **Game** G_1**:** Replace AES-GCM ciphertexts with encryptions of random messages.
- **Game** G_2**:** Replace Paillier ciphertexts with simulated ciphertexts under semantic security.
- **Game** G_3**:** Replace TEE processing with an ideal secure functionality.

The distinguishing advantage between consecutive games is bounded by the security of AES-GCM, Paillier encryption, and TEE confidentiality, respectively. Therefore,

$$|\Pr[G_0 = 1] - \Pr[G_3 = 1]| \leq \epsilon_{AES} + \epsilon_{Paillier} + \epsilon_{TEE}, \quad (3)$$

where all three terms are negligible functions of the security parameter λ.

Since the adversary's advantage in the ideal game G_3 is negligible, the advantage in the real execution is also negligible.

Therefore, sensor readings remain confidential throughout the entire lifecycle of transmission, aggregation, delegation, recovery, and cloud storage.

□

C. Theorem 2: Compartmentalized Key Exposure

Theorem 2. *Let $\mathcal{F}_i$ denote the cryptographic domain associated with fog node F_i.*

Compromise of fog-scoped key k_i affects only $\mathcal{F}_i$ and does not increase the adversarial advantage against any domain $\mathcal{F}_j$ where $j \neq i$.

[$Adv_{\mathcal{A}}^{Comp}(F_j) = 0, \qquad j \neq i.$]

Proof. The KRM provisions an independent AES-GCM key k_i to each fog node F_i.

Sensor assignments satisfy

[$\Gamma(S_j) = F_i$.]

Thus, ciphertexts encrypted under key k_i cannot be decrypted using key k_ℓ where $i \neq \ell$.

Furthermore, the Paillier private key sk_P is never disclosed to any fog node and remains protected by the KRM.

Therefore, compromise of a fog node exposes only the corresponding fog-scoped cryptographic domain and does not affect other domains. Hence compartmentalized key exposure holds.

□

D. Theorem 3: Aggregation Correctness and Integrity

Theorem 3 (Aggregation Correctness and Integrity)**.** *For every aggregation epoch Δ, the aggregate ciphertext $C_{\mathrm{agg},i}$ generated by PLOSHA correctly represents the sum of all sensor readings assigned to fog node F_i. Specifically,*

$$\mathsf{Dec} * sk_P! \, (C * \mathrm{agg}, i) \sum_{k=1}^{m^*} \sum_{j \in \delta_k} d_j. \quad (4)$$

Furthermore, any unauthorized modification of the committed aggregation result is detected with overwhelming probability.

Proof. For each micro-slot δ_k, the TEE enclave computes the encrypted micro-slot aggregate

$$C_{\mathrm{micro},k} \prod_{j \in \delta_k} C_j \pmod{n_P^2}, \quad (5)$$

where $C_j = \mathsf{Enc}_{pk_P}(d_j)$.

By the additive homomorphic property of Paillier encryption,

$$\mathsf{Dec} * sk_P! \, (C * \mathrm{micro}, k) \sum_{j \in \delta_k} d_j. \quad (6)$$

The fog-level aggregate is subsequently obtained as

$$C_{\mathrm{agg},i} \prod_{k=1}^{m^*} C_{\mathrm{micro},k} \pmod{n_P^2}. \quad (7)$$

Combining (6) and (7), the homomorphic property yields

$$\mathsf{Dec} * sk_P! \, (C * \mathrm{agg}, i) \sum_{k=1}^{m^*} \sum_{j \in \delta_k} d_j, \quad (8)$$

which proves aggregation correctness.

To guarantee integrity, the final aggregation package

$$T_i\big(C_i^{\mathrm{final}}, Mode_i, Rel_i, RecStatus_i\big) \quad (9)$$

is digitally signed inside the attested TEE enclave:

$$\sigma_i \mathsf{Sign}_{sk_i^{\mathrm{TEE}}}! \, (H(T_i)) \, . \quad (10)$$

Upon verification, any modification of T_i changes $H(T_i)$ and causes signature verification to fail. Therefore, a malicious adversary cannot alter the aggregation result without being detected, except with negligible probability under the existential unforgeability of the digital signature scheme.

Hence, both aggregation correctness and aggregation integrity are guaranteed. □

E. Theorem 4: Secure Delegation Authenticity

Theorem 4 (Secure Delegation Authenticity)**.** *Assume that remote attestation is secure and that the Key and Reliability Management (KRM) module correctly provisions delegation credentials only to authenticated TEEs.*

Then an aggregation state can be delegated only to an authorized fog node possessing a valid attestation status and a KRM-issued delegation credential. Consequently, an unauthorized fog node cannot impersonate a legitimate delegation recipient except with negligible probability.

Proof. During RMFR recovery, delegation is initiated only after the candidate fog node F_i^* successfully completes remote attestation with the KRM. Specifically, the KRM verifies

$$\mathsf{Attest}(F_i^*) = 1, \tag{11}$$

where a value of 1 indicates that the TEE enclave of F_i^* is authentic and operating in a trusted state.

Upon successful attestation, the KRM generates a temporary delegation credential

$$Cred_i^{\mathrm{del}} \mathsf{GenCred!}\left(F_i^*, \mathsf{Epoch}, \mathsf{Expiry}\right), \tag{12}$$

and securely provisions it to the attested enclave of F_i^*.

Before accepting a delegated aggregation state, the receiving fog node must verify

$$\mathsf{Verify!}\left(Cred_i^{\mathrm{del}}\right) = 1. \tag{13}$$

Any fog node lacking a valid delegation credential fails the verification procedure and is therefore unable to receive or process delegated aggregation states.

Suppose an adversary attempts to impersonate the delegated fog node F_i^*. A successful impersonation requires one of the following events:

1) forging a valid delegation credential;
2) bypassing the remote attestation mechanism; or
3) compromising the trusted TEE enclave.

Under the assumptions of secure credential generation, trustworthy remote attestation, and TEE integrity, each event occurs only with negligible probability.

Therefore, only authorized and remotely attested fog nodes can receive delegated aggregation states, thereby guaranteeing delegation authenticity.

□

F. Theorem 5: Replay Resistance

Theorem 5 (Replay Resistance)**.** *Assuming unique aggregation sequence identifiers and secure TEE verification, the probability that a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ successfully replays a delegation package is negligible. Formally,*

$$\Pr[\mathsf{Replay}_{\mathcal{A}}] \leq \epsilon(\lambda), \tag{14}$$

where $\epsilon(\lambda)$ is a negligible function in the security parameter λ.

Proof. Each delegation package generated by fog node F_i contains a unique aggregation sequence identifier $\mathsf{Seq}_i(t)$ associated with aggregation epoch t.

Let $\mathsf{Seq}_i^{\mathrm{last}}$ denote the most recent sequence identifier previously accepted by the receiving TEE enclave.

Upon receiving a delegation package, the enclave verifies

$$\mathsf{Seq}_i(t) > \mathsf{Seq}_i^{\mathrm{last}}. \tag{15}$$

A package is accepted only if (15) holds. Any replayed delegation message necessarily contains an outdated sequence identifier satisfying

$$\mathsf{Seq}_i(t) \leq \mathsf{Seq}_i^{\mathrm{last}}, \tag{16}$$

and is therefore rejected by the enclave.

Consequently, a successful replay attack requires an adversary to either

1) forge a fresh valid sequence identifier, or
2) compromise the TEE verification mechanism.

Under the assumptions of secure TEE execution and unforgeable sequence generation, both events occur only with negligible probability.

Therefore,

$[\ \Pr[\mathsf{Replay}_{\mathcal{A}}] \leq \epsilon(\lambda),\ ]$

which proves replay resistance. □

G. Theorem 6: Recovery Correctness and Fault-Loss Localization

Theorem 6 (Recovery Correctness and Fault-Loss Localization)**.** *Let D_i^{miss} denote the set of incomplete micro-slots during aggregation epoch Δ and let $C_{\mathrm{agg},i}^{\mathrm{rec}}$ denote the aggregate ciphertext produced by RMFR after recovery.*

If all missing micro-slots are successfully reconstructed, then RMFR preserves aggregation correctness, namely,

$$C_{\mathrm{agg},i}^{\mathrm{rec}} C_{\mathrm{agg},i}. \tag{17}$$

Furthermore, under uniform micro-slot partitioning, the maximum aggregation-loss exposure caused by a single micro-slot failure is bounded by

$$L_{\mathrm{agg}}(m^*)\frac{1}{m^*}, \tag{18}$$

where m^* denotes the number of adaptive micro-slots. Therefore, RMFR localizes recovery to the affected micro-slots and limits aggregation loss to at most $1/m^*$ of the aggregation epoch.

Proof. Let

$$D_i^{\mathrm{valid}} D_i \setminus D_i^{\mathrm{miss}} \tag{19}$$

denote the set of successfully aggregated micro-slots.

During recovery, RMFR reconstructs only the missing micro-slot aggregates and computes the recovered fog-level aggregate as

$$C_{\mathrm{agg},i}^{\mathrm{rec}} \left(\prod_{\delta_k \in D_i^{\mathrm{valid}}} C_{\mathrm{micro},k} \right) \left(\prod_{\delta_k \in D_i^{\mathrm{miss}}} C_{\mathrm{micro},k}^{\mathrm{rec}} \right). \tag{20}$$

If recovery is successful, then every reconstructed micro-slot aggregate satisfies

$$C_{\mathrm{micro},k}^{\mathrm{rec}} C_{\mathrm{micro},k}, \qquad \forall \delta_k \in D_i^{\mathrm{miss}}. \tag{21}$$

Substituting (21) into (20) yields

$$C_{\mathrm{agg},i}^{\mathrm{rec}} \prod_{k=1}^{m^*} C_{\mathrm{micro},k} C_{\mathrm{agg},i}, \tag{22}$$

which proves recovery correctness.

To analyze fault-loss localization, consider an aggregation epoch partitioned into m^* equally sized micro-slots. Each micro-slot contributes

$$\frac{1}{m^*} \tag{23}$$

of the aggregation interval.

Therefore, failure of a single micro-slot affects at most

$$L_{\mathrm{agg}}(m^*)\frac{1}{m^*} \tag{24}$$

of the aggregation epoch.

Unlike conventional flat aggregation, where a late-stage failure may invalidate the entire aggregation window, RMFR recomputes only the affected micro-slots while preserving all valid aggregation results. Consequently, the recovery complexity scales with

$$\mathcal{O}!\left(|D_i^{\mathrm{miss}}|\right) \tag{25}$$

rather than

$$\mathcal{O}(m^*), \tag{26}$$

thereby reducing recovery overhead and improving system resilience.

Hence, RMFR guarantees both aggregation correctness and localized fault recovery.

□

H. Theorem 7: Availability Improvement

Theorem 7 (Availability Improvement)**.** *Let P_{dis} denote the probability that an aggregation epoch is disrupted by overload, communication failure, or fog-node failure.*

Under identical operating conditions, the proposed PLOSHA-RMFR framework achieves lower aggregation-loss exposure and higher service availability than conventional flat aggregation systems. Specifically,

$$L_{\mathrm{PLOSHA}}\frac{1}{m^*} < L_{\mathrm{flat}}, \tag{27}$$

where [$L_{\mathrm{flat}} = 1]and m^* > 1$ is the adaptive number of micro-slots.

Proof. In a conventional flat aggregation framework, all sensor reports collected during an aggregation epoch are combined into a single aggregation unit.

Consequently, if a failure occurs before the aggregation process completes, the entire aggregation epoch becomes unavailable. The corresponding aggregation-loss exposure is

$$L_{\mathrm{flat}}1. \tag{28}$$

In contrast, PLOSHA partitions the aggregation epoch into m^* adaptive micro-slots and performs hierarchical aggregation over individual micro-slot aggregates.

From Theorem 6, the maximum aggregation-loss exposure caused by a single micro-slot failure is bounded by

$$L_{\mathrm{PLOSHA}}\frac{1}{m^*}. \tag{29}$$

Since

$$m^* > 1, \tag{30}$$

it follows directly that

$$L_{\mathrm{PLOSHA}} < L_{\mathrm{flat}}. \tag{31}$$

Furthermore, RMFR employs a progressive recovery strategy consisting of:

1) predictive delegation for anticipated overloads;
2) selective micro-slot recovery for incomplete aggregation;
3) reliability-aware failover for unavailable fog nodes.

These mechanisms reduce the probability that a disruption results in complete aggregation failure. Let $P_{\mathrm{dis}}^{\mathrm{RMFR}}$ and $P_{\mathrm{dis}}^{\mathrm{flat}}$ denote the disruption probabilities of PLOSHA-RMFR and a conventional flat aggregation framework, respectively.

Because RMFR provides proactive delegation, localized recovery, and failover continuity,

$$P_{\mathrm{dis}}^{\mathrm{RMFR}} < P_{\mathrm{dis}}^{\mathrm{flat}}. \tag{32}$$

Therefore, PLOSHA-RMFR simultaneously reduces aggregation-loss exposure and service-disruption probability, resulting in improved operational availability.

□

I. Theorem 8: Stability of AFLTO Optimization

Theorem 8 (Bounded Stability of AFLTO)**.** *Let $\tau_x(t)$ denote any adaptive threshold maintained by the AFLTO module, where*

[$\tau_x \in \tau_v, \tau_r, \tau_1, \tau_2, \tau_3, \tau_f.$]

Assume that the feedback error $e_i(t)$ is bounded such that

$$|e_i(t)| \le e_{\max}, \tag{33}$$

and that the learning rate satisfies

$$0 < \mu_x < 1. \tag{34}$$

Then all AFLTO thresholds remain bounded within the interval $[0, 1]$ and converge toward a stable operating region without divergence.

Proof. The AFLTO update rule is defined as

$$\tau_x(t+1)\Pi_{[0,1]}!\left(\tau_x(t) + \mu_x e_i(t)\right), \tag{35}$$

where

$$\Pi_{[0,1]}(y)\min 1, \max\left(0, y\right) \tag{36}$$

denotes the projection operator onto the closed interval $[0, 1]$.

By definition of the projection operator,

$$0 \le \tau_x(t+1) \le 1 \tag{37}$$

for every iteration t, regardless of the value of $e_i(t)$.

Therefore, the threshold sequence cannot diverge outside the feasible operating range.

Furthermore, from (33) and (35), the maximum threshold variation between two consecutive iterations satisfies

$$|\tau_x(t+1) - \tau_x(t)| \leq \mu_x e_{\max}. \tag{38}$$

Since both μ_x and $e_{\max}$ are finite, threshold updates remain bounded and cannot exhibit unbounded growth.

AFLTO additionally enforces the recovery-escalation ordering

$$\tau_1 < \tau_2 < \tau_3, \tag{39}$$

which guarantees consistent transition among predictive delegation, localized recovery, and failover recovery modes.

Consequently, all adaptive thresholds remain bounded, non-divergent, and operationally consistent throughout long-term system execution.

Hence, AFLTO achieves bounded stability.

□

This section evaluates the efficiency, scalability, and fault-resilience of the proposed PLOSHA-RMFR framework. The evaluation consists of three parts: computation cost analysis, communication cost analysis, and performance analysis. To provide a comprehensive comparison, four representative schemes are selected from the related work, including FedDQN [22], Robust IIoT Aggregation [24], Fault-Tolerant Workflow Scheduling [37], and SIREN [38]. These schemes represent learning-based workload adaptation, privacy-preserving encrypted aggregation, fault-tolerant scheduling and recovery, and optimization-based resource management, respectively. Together, they cover the key functionalities addressed by PLOSHA-RMFR and provide suitable baselines for evaluating predictive adaptation, secure aggregation, and fault recovery in IIoT edge–fog environments.

J. Computation Cost Analysis

This subsection analyzes the computation cost of PLOSHA-RMFR and compares it with the selected representative schemes. The analysis considers the major operations performed during predictive workload estimation, scheduling, encrypted aggregation, recovery, and secure delegation. In particular, the proposed framework combines lightweight EWMA-based prediction, TEE-assisted ciphertext transformation, Paillier homomorphic aggregation, and localized micro-slot recovery within a unified aggregation architecture.

Let N_s denote the number of sensor reports collected during an aggregation epoch, m^* denote the adaptive number of aggregation micro-slots, and $|D_i^{miss}|$ denote the number of incomplete micro-slots requiring recovery. The major computational primitives used throughout the analysis are summarized in Table III, while the overall computation-cost comparison is presented in Table IV.

Table IV compares the computation cost of PLOSHA-RMFR with representative workload management, fault-tolerant scheduling, and privacy-preserving aggregation schemes. Ref. [22] incurs additional overhead from DQN training and federated model aggregation, while Ref. [24] relies on computationally intensive cryptographic operations, including Paillier encryption, signature generation, and batch verification. Ref. [37] performs fault-tolerant scheduling through replication and resubmission, resulting in scheduling and recovery costs that increase with workflow size and resource configurations. Ref. [38] employs MT-LSTM prediction and graph-based placement optimization, leading to relatively high computational complexity.

In contrast, PLOSHA-RMFR combines lightweight EWMA-based prediction, secure aggregation, and localized recovery. Although the framework introduces additional cryptographic operations through TEE-assisted Paillier aggregation, sensor devices perform only lightweight AES-GCM encryption. More importantly, the recovery cost scales with the number of incomplete micro-slots, $|D_i^{miss}|$, rather than the entire processing window. Since typically $|D_i^{miss}| \ll m^*$, PLOSHA-RMFR significantly reduces recomputation overhead while maintaining secure and fault-resilient aggregation in large-scale IIoT edge–fog environments.

TABLE III: Computation Cost Notations

Notation	Description
Cryptographic and Recovery Operations	
T_{AES}	AES-GCM encryption/decryption
T_{PEnc}	Paillier encryption
T_{PAdd}	Paillier homomorphic aggregation
T_{PDec}	Paillier decryption
T_{TEE}	TEE-assisted ciphertext transformation
T_{Sig}	Digital signature generation
T_{BVer}	Batch signature verification
T_{Noise}	Differential-privacy noise generation
Prediction and Scheduling Operations	
T_{pred}	EWMA-based workload prediction
T_{risk}	Risk assessment and recovery decision
T_{sch}	Scheduling/load-balancing operation
T_{KM}	K-means clustering operation
T_{DQN}	Deep-Q network training/inference
T_{FedAgg}	Federated model aggregation
T_{fluc}	Resource-fluctuation estimation
System Parameters	
N_s	Number of sensor reports
m^*	Number of adaptive micro-slots
$\lvert D_i^{miss}\rvert$	Number of incomplete micro-slots
J	Number of workflow tasks
N_{in}	Maximum task indegree
M	Number of VM types
N_M	Maximum number of active VMs
T	Number of time slots
$\lvert R\rvert$	Number of service requests
$\lvert V\rvert$	Number of cloudlets/nodes
$\lvert E\rvert$	Number of network links
$\lvert V_m\rvert$	Number of functions in request m
g_m	Number of MT-LSTM groups
h_g	Hidden-state dimension of MT-LSTM
Θ	$\lvert V_m\rvert\lvert V\rvert \log(\lvert V_m\rvert\lvert V\rvert) + \lvert V_m\rvert(\lvert E\rvert + \lvert V\rvert)$

K. Communication Cost Analysis

This subsection analyzes the communication overhead of PLOSHA-RMFR and compares it with representative workload-management, fault-tolerant scheduling, and privacy-preserving aggregation schemes. The analysis focuses on the communication incurred during data collection, coordination, aggregation, and recovery processes. In particular, the pro-

TABLE IV: Computation Cost Comparison

Scheme	Prediction	Scheduling	Aggregation	Recovery
Ref. [22]	T_{DQN}	$N_s T_{KM}$	T_{FedAgg}	–
Ref. [24]	–	–	$N_s(T_{PEnc} + T_{Sig} + T_{BVer} + T_{PAdd})$	–
Ref. [37]	T_{fluc}	$O(J^2 + (J + N_{in})MN_M)$	–	$O((J + N_{in})MN_M + JN_{in})$
Ref. [38]	$O(T\|R\|g_m h_g^2)$	$O(\|R\| \log \|V\| \Theta)$	–	$O(\|R\| \log \|V\| \Theta)$
PLOSHA-RMFR	$T_{pred} + T_{risk}$	$N_s T_{sch}$	$N_s(T_{AES} + T_{TEE} + T_{PEnc} + T_{PAdd})$	$\|D_i^{miss}\| T_{PAdd}$

Note: $|D_i^{miss}|$ denotes the number of incomplete micro-slots requiring recovery. Unlike Refs. [24], [37], [38], whose recovery or reconfiguration overhead scales with the entire processing window, PLOSHA-RMFR performs localized recovery only on affected micro-slots, resulting in recovery complexity proportional to $|D_i^{miss}|$, where typically $|D_i^{miss}| \ll m^*$.

TABLE V: Communication Cost Comparison

Scheme	Data Collection	Coordination	Recovery
Ref. [22]	$N_s\|Data\|$	$\|Model\|$	–
Ref. [24]	$N_s(\|CT_P\| + \|Sig\|)$	$N_s\|Sig\|$	–
Ref. [37]	$N_s\|Data\|$	$\|Sched\|$	$m^*\|State\|$
Ref. [38]	$N_s\|Data\|$	$\|Pred\| + \|Place\|$	$m^*\|State\|$
PLOSHA-RMFR	$N_s\|CT_{AES}\|$	$m^*\|Meta\|$	$\|D_i^{miss}\|(\|DSP\| + \|FSM\|)$

Note: $|CT_{AES}|$ and $|CT_P|$ denote AES-GCM and Paillier ciphertext sizes, respectively; $|Meta|$ denotes micro-slot metadata; $|DSP|$ denotes delegation-state packets; $|FSM|$ denotes failover-state migration data. Unlike Refs. [37], [38], whose recovery communication scales with the entire processing window, PLOSHA-RMFR performs localized recovery and exchanges recovery information only for affected micro-slots, yielding communication complexity proportional to $|D_i^{miss}|$, where typically $|D_i^{miss}| \ll m^*$.

posed framework employs encrypted sensor-to-fog transmission, adaptive micro-slot coordination, and localized recovery mechanisms to reduce communication overhead while maintaining secure and fault-resilient aggregation.Table V presents the communication-cost comparison among the selected schemes.

Table V shows that the communication overhead of PLOSHA-RMFR remains lightweight and scalable compared with existing approaches. Ref. [22] incurs additional communication due to federated model synchronization, whereas Ref. [24] requires the transmission of large Paillier ciphertexts and digital signatures for secure aggregation and verification. Refs. [37] and [38] further introduce substantial state-transfer overhead during workflow recovery, service migration, and standby reconfiguration, causing recovery communication to scale with the entire processing window. In contrast, PLOSHA-RMFR employs compact AES-GCM ciphertexts for sensor-to-fog communication and exchanges only lightweight metadata for micro-slot coordination. More importantly, its localized recovery mechanism limits communication to affected micro-slots, resulting in recovery overhead proportional to $|D_i^{miss}|$ rather than the full aggregation window size m^*. Consequently, PLOSHA-RMFR achieves lower communication overhead while maintaining secure aggregation, adaptive coordination, and fault resilience in large-scale IIoT edge–fog environments.

“‘latex

L. Performance Analysis

This subsection presents the experimental design used to evaluate the runtime behavior of PLOSHA-RMFR under dynamic IIoT edge–fog conditions. The objective is to assess whether the proposed framework can improve aggregation efficiency, reduce recovery overhead, and maintain service availability under workload fluctuation, incomplete aggregation, and fog-node failures.

1) Experimental Setup: The evaluation considers an edge–fog IIoT environment consisting of multiple sensors, fog nodes, and a cloud server. Sensors continuously generate time-series industrial measurements and transmit encrypted reports to their assigned fog nodes. Each fog node performs adaptive micro-slot aggregation and invokes RMFR when overload, incomplete aggregation, or failure is detected.

The experiments compare PLOSHA-RMFR with four representative schemes: Ref. [22], Ref. [24], Ref. [37], and Ref. [38]. Ref. [22] represents learning-based adaptive scheduling, Ref. [24] represents privacy-preserving aggregation, Ref. [37] represents hybrid fault-tolerant workflow scheduling, and Ref. [38] represents fault-tolerant serverless edge stream processing.

Unless otherwise stated, the number of sensors is varied from 500 to 5000, the number of fog nodes is varied from 5 to 50, and each aggregation epoch is divided into adaptive micro-slots. The failure rate is varied from 2% to 20%, and the workload intensity is varied by changing the sensor reporting rate. Each experiment is repeated multiple times, and the average value is reported.

The main evaluation metrics are summarized as follows:

- **Aggregation latency**: the time required to complete one aggregation epoch.
- **Recovery latency**: the time required to restore incomplete or failed aggregation.
- **Aggregation completeness**: the ratio of received reports to expected reports.
- **Aggregation-loss exposure**: the fraction of aggregation results affected by failure.

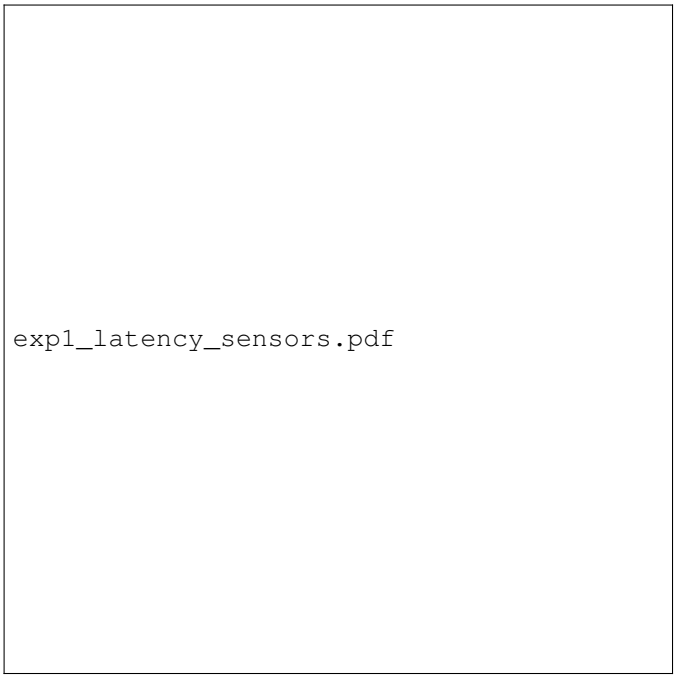

Fig. 2: Aggregation latency comparison under different numbers of sensors.

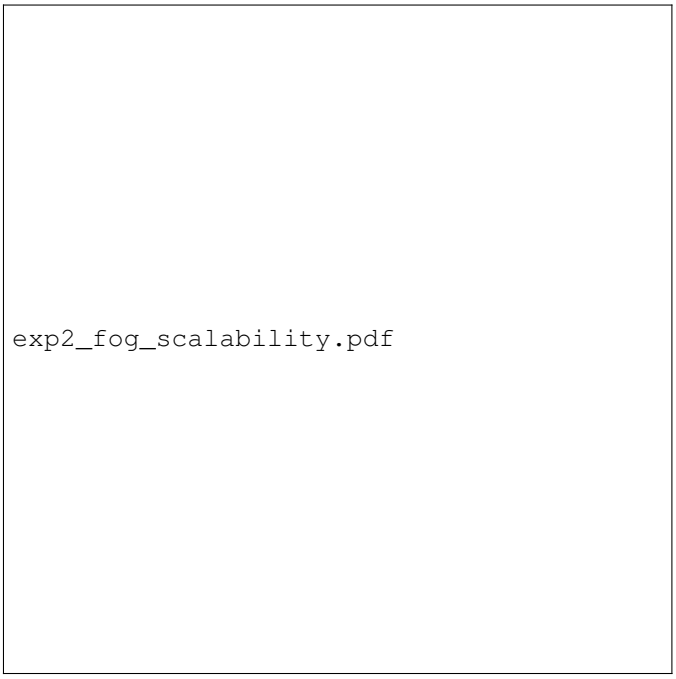

Fig. 3: Aggregation latency versus number of fog nodes.

- **Communication overhead**: the amount of data exchanged during aggregation and recovery.
- **System availability**: the ratio of successfully completed aggregation epochs.

Experiment 1: Scalability With Number of Sensors: This experiment evaluates aggregation scalability under increasing IIoT data volume. The number of sensors is varied from 500 to 5000 while the number of fog nodes, aggregation epoch length, and network settings are fixed. PLOSHA-RMFR is compared with Refs. [22], [24], [37], and [38] using aggregation latency as the main metric. For fairness, all schemes process the same number of sensor reports in each epoch. The results are shown in Fig. 2.

As shown in Fig. 2, aggregation latency increases for all schemes as the number of sensors grows. Ref. [24] incurs higher latency because it performs flat privacy-preserving aggregation over encrypted reports. Refs. [22], [37], and [38] mainly optimize scheduling or service placement, but they do not provide an adaptive encrypted aggregation structure, causing latency to increase under dense sensor workloads. In contrast, PLOSHA-RMFR achieves lower latency by partitioning each epoch into adaptive micro-slots and performing hierarchical aggregation at the fog layer. This reduces aggregation bottlenecks and distributes processing more efficiently. Therefore, the results demonstrate that PLOSHA-RMFR scales better than the selected baselines as IIoT sensor density increases.

Experiment 2: Scalability With Number of Fog Nodes: This experiment evaluates the scalability of PLOSHA-RMFR as the number of fog nodes increases. The number of fog nodes is varied from 5 to 50 while maintaining a fixed number of sensors and aggregation workload. The objective is to examine the ability of the proposed framework to distribute aggregation and recovery workloads across heterogeneous fog resources. PLOSHA-RMFR is compared with Refs. [22], [37], and [38], which also employ fog- or edge-assisted processing architectures. The evaluation metric is the average aggregation latency per epoch. The results are shown in Fig. 3.

As shown in Fig. 3, the aggregation latency decreases for all schemes as additional fog nodes become available because the workload can be distributed across more computational resources. However, PLOSHA-RMFR consistently achieves the lowest latency throughout the evaluation range. The improvement is primarily attributed to the PAHSA predictive estimation mechanism, which dynamically evaluates fog-node capacity, risk, and workload conditions before assigning aggregation tasks. Furthermore, RMFR proactively delegates workloads from potentially overloaded nodes and localizes recovery operations when failures occur.

In contrast, Ref. [22] improves resource utilization through federated reinforcement learning but incurs additional decision overhead associated with model updates and synchronization. Ref. [37] relies on replication and resubmission mechanisms, resulting in higher scheduling and recovery overhead under dynamic workload conditions. Ref. [38] employs MT-LSTM prediction and graph-based placement optimization; however, frequent placement adjustments introduce additional processing overhead as the edge infrastructure grows. Consequently, PLOSHA-RMFR exhibits better scalability and maintains lower aggregation latency as the number of fog nodes increases, demonstrating its effectiveness for large-scale IIoT edge–fog deployments.

Experiment 3: Impact of Workload Intensity: This experiment evaluates the robustness of PLOSHA-RMFR under varying workload conditions. The sensor reporting rate

Fig. 4: Impact of workload intensity on aggregation latency.

Fig. 5: Recovery latency versus fog-node failure rate.

is gradually increased by reducing the reporting interval, thereby generating low, medium, and high workload scenarios. PLOSHA-RMFR is compared with Refs. [22], [24], [37], and [38]. The evaluation metrics include aggregation latency, queue utilization, and recovery frequency. The results are shown in Fig. 4.

As shown in Fig. 4, the aggregation latency of all schemes increases as workload intensity grows due to the larger number of sensor reports processed within each aggregation epoch. Ref. [24] exhibits the steepest increase because encrypted aggregation operations accumulate rapidly under high reporting rates. Ref. [22] adapts workload distribution through federated reinforcement learning; however, model synchronization and learning overhead become more pronounced as the workload increases. Ref. [37] relies on replication and resubmission mechanisms that introduce additional scheduling overhead during workload surges, while Ref. [38] incurs repeated placement-adjustment costs associated with stream-processing optimization.

In contrast, PLOSHA-RMFR consistently maintains lower aggregation latency across all workload levels. The PAHSA mechanism dynamically adjusts aggregation granularity according to predicted processing capacity and workload conditions, enabling aggregation tasks to be distributed across adaptive micro-slots. Furthermore, RMFR proactively delegates workloads from overloaded fog nodes before failures occur, reducing queue congestion and preventing excessive recovery operations. As a result, the increase in latency remains significantly lower than that of the baseline schemes. These results demonstrate that PLOSHA-RMFR effectively adapts to dynamic workload fluctuations and maintains stable aggregation performance in large-scale IIoT environments.

***Experiment 4: Impact of Failure Rate**:* This experiment evaluates the fault resilience of PLOSHA-RMFR under increasing fog-node failures. The failure rate is varied from 2% to 20% by randomly introducing node outages during aggregation epochs. PLOSHA-RMFR is compared with Refs. [22], [37], and [38]. The primary evaluation metric is recovery latency, while aggregation completeness and system availability are used as secondary metrics. The results are shown in Fig. 5.

As shown in Fig. 5, the recovery latency of all schemes increases as the failure rate grows because more tasks, aggregation states, or service instances require restoration. However, PLOSHA-RMFR consistently achieves the lowest recovery latency across all failure levels. The improvement is primarily attributed to the RMFR framework, which localizes recovery to incomplete micro-slots and performs selective reconstruction only for affected aggregation regions.

In contrast, Ref. [37] employs replication and task resubmission mechanisms that recover larger portions of the workflow, resulting in higher recovery overhead as failures increase. Ref. [38] relies on service relocation and standby reconfiguration, which require additional placement and migration operations before normal execution can resume. Although Ref. [22] adapts workload distribution through federated learning, it does not explicitly optimize aggregation recovery and therefore experiences increased latency under severe failures.

Furthermore, PLOSHA-RMFR maintains higher aggregation completeness and system availability because predictive delegation can proactively transfer workloads from high-risk fog nodes before failures occur. Consequently, the proposed framework confines recovery overhead to affected micro-slots, causing recovery complexity to scale with $|D_i^{miss}|$ rather than the entire aggregation window size m^*. These results demonstrate that PLOSHA-RMFR provides superior fault tol-

Fig. 6: Aggregation-loss exposure versus number of micro-slots.

Fig. 7: Recovery communication overhead versus number of incomplete micro-slots.

erance and recovery efficiency for large-scale IIoT edge–fog environments.

Experiment 5: Aggregation-Loss Exposure: This experiment evaluates the ability of PLOSHA-RMFR to reduce aggregation-loss exposure under failure conditions. The objective is to quantify the fraction of aggregation results affected when failures occur during an aggregation epoch. The number of adaptive micro-slots is varied from 1 to 20 while maintaining a fixed workload and failure rate. PLOSHA-RMFR is compared with Ref. [24], Ref. [37], and Ref. [38]. The evaluation metric is aggregation-loss exposure, defined as the proportion of aggregation data that becomes unavailable or requires recomputation following a failure. The results are shown in Fig. 6.

As shown in Fig. 6, PLOSHA-RMFR consistently achieves the lowest aggregation-loss exposure among all compared schemes. Ref. [24] performs flat encrypted aggregation, causing a single failure to potentially affect the entire aggregation epoch. Similarly, Refs. [37] and [38] recover failures at the workflow or service level, resulting in larger recovery scopes and greater aggregation disruption. In contrast, PLOSHA-RMFR partitions each aggregation epoch into adaptive micro-slots and confines failures to affected aggregation regions. Consequently, only a small fraction of the aggregation result is exposed when a failure occurs.

The results further show that aggregation-loss exposure decreases as the number of micro-slots increases. According to the PAHSA design, the maximum loss exposure is bounded by

$$L_{\mathrm{PLOSHA}} = \frac{1}{m^*},$$

where m^* denotes the number of adaptive micro-slots. Therefore, increasing m^* improves fault isolation and reduces the amount of aggregation data affected by individual failures. These results confirm that the proposed hierarchical aggregation architecture significantly improves aggregation resilience and minimizes the impact of failures in large-scale IIoT environments.

Experiment 6: Recovery Communication Overhead: This experiment evaluates the communication overhead incurred during fault recovery. The number of incomplete micro-slots is varied while maintaining a fixed aggregation workload and failure rate. PLOSHA-RMFR is compared with Refs. [37] and [38], which perform recovery through workflow resubmission, task replication, service migration, or standby reconfiguration. The evaluation metric is the amount of recovery-related data exchanged between fog nodes, measured in kilobytes (KB). The results are shown in Fig. 7.

As shown in Fig. 7, the recovery communication overhead increases for all schemes as the number of incomplete aggregation regions grows. However, PLOSHA-RMFR consistently requires significantly less communication than the baseline schemes. The improvement is primarily attributed to the RMFR localized recovery mechanism, which exchanges recovery information only for affected micro-slots and avoids retransmitting complete aggregation windows.

In contrast, Ref. [37] performs workflow-level recovery through replication and task resubmission, requiring additional state-transfer and scheduling messages as failures increase. Ref. [38] incurs even higher communication overhead because service migration and standby reconfiguration require transferring larger execution states and placement information across distributed cloudlets. Consequently, the recovery communication overhead of both schemes grows more rapidly with

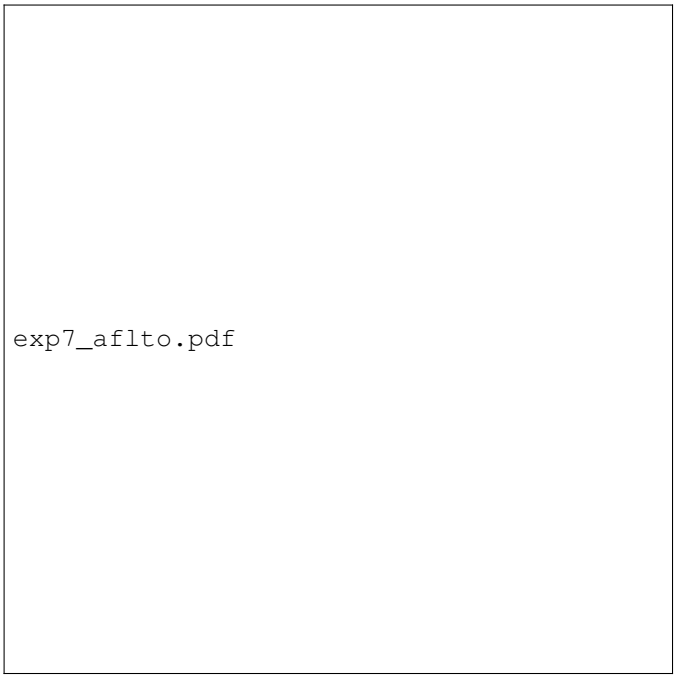

Fig. 8: Impact of AFLTO on aggregation completeness and system availability.

increasing failure severity.

By restricting recovery communication to incomplete micro-slots, PLOSHA-RMFR achieves recovery complexity proportional to $|D_i^{miss}|$ rather than the full aggregation window size m^*. Therefore, even under frequent failures, the amount of exchanged recovery data remains substantially lower than that of the baseline schemes. These results confirm that the proposed framework improves communication efficiency while preserving fault resilience in large-scale IIoT edge–fog environments.

***Experiment 7: Effectiveness of AFLTO**:* This experiment evaluates the contribution of the Adaptive Feedback Learning and Threshold Optimization (AFLTO) mechanism. An ablation study is conducted by comparing the complete PLOSHA-RMFR framework with a variant in which AFLTO is disabled and all aggregation and recovery thresholds remain static throughout system operation. The workload intensity and fog-node failure rate are dynamically varied over time to emulate realistic IIoT environments with changing operational conditions. The evaluation metrics include aggregation completeness, recovery frequency, and system availability. The results are shown in Fig. 8.

As shown in Fig. 8, the complete PLOSHA-RMFR framework consistently achieves higher aggregation completeness and system availability than the static-threshold variant. When workload intensity and failure conditions change, fixed thresholds cannot adapt to evolving operating environments, causing either excessive recovery operations or delayed recovery activation. Consequently, aggregation completeness decreases and system performance becomes less stable over time.

In contrast, AFLTO continuously adjusts aggregation and recovery thresholds according to observed aggregation quality, reliability trends, and recovery effectiveness. By incorporating both short-term operational feedback and historical performance information, AFLTO enables the framework to respond proactively to workload fluctuations and reliability degradation. As a result, unnecessary recovery operations are reduced, aggregation completeness is improved, and service availability remains consistently high.

Furthermore, the results demonstrate that AFLTO stabilizes long-term system behavior by maintaining threshold values within appropriate operating ranges while preserving the recovery escalation strategy defined by RMFR. These findings confirm that the proposed feedback optimization mechanism significantly enhances the adaptability and robustness of PLOSHA-RMFR under dynamic IIoT edge–fog conditions.

REFERENCES

[1] K. Tange, M. De Donno, X. Fafoutis, and N. Dragoni, "A systematic survey of industrial Internet of Things security: Requirements and fog computing opportunities," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 4, pp. 2489–2520, 2020.

[2] M. Alshammeri, M. Humayun, K. Haseeb, M. Alamri, A. Chehri and G. Jeon, "Intent-Based Secure Fault Tolerance Model With Integrated AI for Edge-IoT Networks," in IEEE Internet of Things Journal, vol. 13, no. 9, pp. 18325-18333

[3] J. Jin, K. Yu, J. Kua, N. Zhang, Z. Pang, and Q.-L. Han, "Cloud-fog automation: Vision, enabling technologies, and future research directions," *IEEE Trans. Ind. Informat.*, vol. 20, no. 2, pp. 1039–1054, 2024.

[4] M. H. Kashani and E. Mahdipour, "Load balancing algorithms in fog computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 2, pp. 1505–1521, 2023.

[5] S. Chen *et al.*, "A hybrid approach for fault-tolerance aware load balancing in fog computing," *Cluster Comput.*, vol. 27, pp. 1–20, 2024.

[6] Y. Lu, Y. Zhang, Y. Zheng, R. Zhu and W. Xiang, "Adaptive Gossip-Enhanced SIR Models for Real-Time Routing Optimization and Fault Tolerance in Distributed Networks," in IEEE Transactions on Network and Service Management, vol. 23, pp. 3819-3832, 2026, doi: 10.1109/TNSM.2026.3680838.

[7] C. Peng, M. Luo, P. Vijayakumar, D. He, O. Said, and A. Tolba, "Multifunctional and multidimensional secure data aggregation scheme in WSNs," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2657–2668, 2022.

[8] M. Vijarania *et al.*, "A systematic literature review on load-balancing techniques in fog computing: Architectures, strategies, and emerging trends," *Computers*, vol. 14, no. 6, p. 217, 2025.

[9] A. Nemati and N. Mansouri, "Resource allocation in fog computing: A survey on current state and research challenges," *Knowl. Inf. Syst.*, 2024.

[10] N. Tripathy, S. Sahoo, N. S. Alghamdi *et al.*, "Energy and makespan optimised task mapping in fog enabled IoT application: A hybrid approach," *Sci. Rep.*, vol. 16, p. 5210, 2026.

[11] M. Kaur and R. Aron, "FOCALB: Fog computing architecture of load balancing for scientific workflow applications," *J. Grid Comput.*, vol. 19, no. 4, p. 40, 2021.

[12] M. Ebrahim and A. Hafid, "Resilience and load balancing in fog networks: A multi-criteria decision analysis approach," *Microprocess. Microsyst.*, vol. 101, p. 104893, 2023.

[13] A. Ahmed, A. Alsharkawy, M. E. Embabi, and A. Emara, "Adaptive multi-criteria-based load balancing technique for resource allocation in fog-cloud environments," *Int. J. Comput. Netw. Commun.*, vol. 16, no. 1, pp. 105–124, 2024.

[14] G. K. Walia, M. Kumar, and S. S. Gill, "AI-empowered fog/edge resource management for IoT applications: A comprehensive review, research challenges, and future perspectives," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 1, pp. 619–669, 2024.

[15] A. M. Jasim and H. Al-Raweshidy, "An Adaptive SDN-Based Load Balancing Method for Edge/Fog-Based Real-Time Healthcare Systems," in IEEE Systems Journal, vol. 18, no. 2, pp. 1139-1150, June 2024, doi: 10.1109/JSYST.2024.3402156.

[16] A. Mukhopadhyay and M. Ruffini, "Edge Server Load Balancing Using Steerable Free Space Optics for Partial Mesh Optical Access Networks," in IEEE Transactions on Communications, vol. 74, pp. 4853-4865, 2026, doi: 10.1109/TCOMM.2026.3664452.

[17] C. Tian, H. Cao, J. Xie, S. Garg, M. Alrashoud and P. Tiwari, "Community Detection-Empowered Self-Adaptive Network Slicing in Multi-Tier Edge-Cloud System," in IEEE Transactions on Network and Service Management, vol. 21, no. 3, pp. 2624-2636, June 2024, doi: 10.1109/TNSM.2023.3332509.
[18] M. Kirti, A. K. Maurya, and R. S. Yadav, "Fault-tolerance approaches for distributed and cloud computing environments: A systematic review, taxonomy and future directions," *Concurrency Comput. Pract. Exper.*, vol. 36, no. 13, p. e8081, 2024.
[19] F. Liu, K. Hu, J. He, W. Hu, H. Li, M. Peng, and Y. He, "A fault-tolerant scheduling algorithm that minimizes the number of replicas in heterogeneous service-oriented cloud computing systems," *J. Supercomput.*, vol. 80, no. 9, pp. 13079–13095, 2024.
[20] H. T. Rajab and M. F. Younis, "Dynamic fault tolerance aware scheduling for healthcare system on fog computing," *Iraqi J. Sci.*, vol. 62, no. 1, pp. 308–318, 2021.
[21] V. Mohammadi, A. M. Rahmani, A. Darwesh, and A. Sahafi, "Fault tolerance in fog-based Social Internet of Things," *Knowl.-Based Syst.*, vol. 265, p. 110376, 2023.
[22] P. Choppara and S. S. Mangalampalli, "Adaptive Task Scheduling in Fog Computing Using Federated DQN and K-Means Clustering," in IEEE Access, vol. 13, pp. 75466-75492, 2025, doi: 10.1109/ACCESS.2025.3563487.
[23] M. R. Saha, M. Ehsanul Kader and R. Reaz, "Dependency, Deadline and Priority Aware Multi-Queue Dynamic Task Scheduling Using Heterogeneous Resources in Fog Environment," 2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC), Sharjah, United Arab Emirates, 2024, pp. 9-16, doi: 10.1109/UCC63386.2024.00012.
[24] S. Shang, X. Li, K. Gu, L. Li, X. Zhang, and V. Pandi, "A robust privacy-preserving data aggregation scheme for edge-supported IIoT," *IEEE Trans. Ind. Informat.*, vol. 20, no. 3, pp. 4305–4416, 2024.
[25] A. Younesi, M. Toghani, S. Safari, M. Ansari and T. Fahringer, "MOSAIC: Mobility-Oriented Scheduling and Intelligent Resource Allocation for IoT," in IEEE Transactions on Mobile Computing, vol. 25, no. 5, pp. 7291-7307, May 2026, doi: 10.1109/TMC.2025.3640765.
[26] Y. Zheng, L. Hua, Y. Li and H. Lu, "Adaptive Network Slicing and Hierarchical Data Management for Edge-Assisted Multi-Task Video Enhancement," in IEEE Network, vol. 39, no. 3, pp. 63-71, May 2025, doi: 10.1109/MNET.2025.3535810.
[27] Z. Qin, H. Xiong, S. Wu, and J. Batamuliza, "A survey of proxy re-encryption for cloud data sharing," *IEEE Trans. Serv. Comput.*, vol. 15, no. 4, pp. 1964–1982, 2022.
[28] M. Kumar *et al.*, "AI-based sustainable and intelligent offloading framework for IIoT in collaborative cloud-fog environments," *IEEE Trans. Consum. Electron.*, 2023.
[29] M. Abdel-Basset, G. Manogaran, M. Mohamed, and E. Rushdy, "A fault-tolerant aware scheduling method for fog-cloud environments," *Future Gener. Comput. Syst.*, vol. 93, pp. 1–12, 2019.
[30] A. Demers *et al.*, "Epidemic algorithms for replicated database maintenance," *ACM SIGOPS Oper. Syst. Rev.*, vol. 22, no. 1, pp. 8–32, 1988.
[31] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st ACM MCC Workshop Mobile Cloud Comput.*, 2012, pp. 13–16.
[32] A. Younesi, M. Ansari, A. Ejlali, M. A. Fazli, M. Shafique and J. Henkel, "SIREN: Multiobjective Game-Theoretic Scheduler Based on Memory-Driven Gray Wolf Optimization in Fog–Cloud Computing," in IEEE Internet of Things Journal, vol. 13, no. 11, pp. 22983-22999, 1 June1, 2026, doi: 10.1109/JIOT.2026.3666558.
[33] A. Uta and F. J. Seinstra, "P2-SWAN: Privacy-preserving stream processing in the fog," in *Proc. IEEE Int. Conf. Fog Edge Comput. (ICFEC)*, 2017, pp. 1–10.
[34] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Advances in Cryptology–EUROCRYPT'99*, LNCS, vol. 1592, 1999, pp. 223–238.
[35] R. Liu, W. Wu, X. Guo, G. Zeng, and K. Li, "Replica fault-tolerant scheduling with time guarantee under energy constraint in fog computing," *Future Gener. Comput. Syst.*, vol. 159, pp. 567–579, 2024.
[36] M. H. Shahab, Y. Sharma, A. Jindal, and A. Al-Dulaimy, "A bi-objective policy for resilient and sustainable SFC management in telco-cloud environments," *IEEE Access*, vol. 13, 2025.
[37] Q. Ren and G. Yao, "A Hybrid Fault-Tolerant Workflow Scheduling With Performance Fluctuated Cloud Resources," *IEEE Trans. Serv. Comput.*, vol. 19, no. 1, pp. 44–57, Jan.–Feb. 2026.
[38] Z. Xu *et al.*, "Efficient and Fault Tolerant Data Stream Processing With Uncertain Data Rates in Serverless Edge Computing," *IEEE Trans. Serv. Comput.*, vol. 19, no. 1, pp. 295–308, Jan.–Feb. 2026.
[39] K. Cao, C. Tan, Y. Cui, and K. Li, "Preference-Aware Fault-Tolerant Function Embedding in Energy-Harvesting Serverless Edge Computing," *IEEE Trans. Serv. Comput.*, vol. 19, no. 2, pp. 1464–1477, Mar.–Apr. 2026.
[40] A. Taghinezhad-Niar and J. Taheri, "Fault-Tolerant Cost-Efficient Scheduling for Energy and Deadline-Constrained IoT Workflows in Edge-Cloud Continuum," *IEEE Trans. Serv. Comput.*, vol. 18, no. 5, pp. 2892–2903, Sept.–Oct. 2025.
[41] Q. He *et al.*, "EdgeHydra: Fault-Tolerant Edge Data Distribution Based on Erasure Coding," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 1, pp. 29–42, Jan. 2025.
[42] S. Long *et al.*, "Fault-Tolerant Aware Task Offloading Based on Reinforcement Learning in Mobile Edge Computing," *IEEE Trans. Mobile Comput.*, vol. 25, no. 5, pp. 6068–6082, May 2026.